

UNIVERSITÀ DEGLI STUDI DI PERUGIA  
Dipartimento di Matematica e Informatica



A.D. 1308  
**unipg**  
DIPARTIMENTO  
DI MATEMATICA E INFORMATICA

TESI MAGISTRALE IN INFORMATICA

# Sviluppo di un Covert Channel tramite il protocollo ICMP per l'esfiltrazione di dati

*Relatore*

**Prof. Santini Francesco**

*Laureando*

**Mecarelli Marco**

---

Anno Accademico 2024-2025

# Indice

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                   | <b>7</b>  |
| <b>2</b> | <b>Covert Channel e Protocollo ICMP</b>               | <b>9</b>  |
| 2.1      | Internet Control Message Protocol . . . . .           | 9         |
| 2.1.1    | Tipologie di Messaggi ICMP . . . . .                  | 11        |
| 2.2      | Covert Channel . . . . .                              | 18        |
| 2.2.1    | Tipologie di Covert Channel . . . . .                 | 20        |
| <b>3</b> | <b>Strumenti Utilizzati</b>                           | <b>21</b> |
| <b>4</b> | <b>Implementazione</b>                                | <b>25</b> |
| 4.1      | Struttura della comunicazione fra le entità . . . . . | 27        |
| 4.1.1    | Struttura dell'attaccante . . . . .                   | 30        |
| 4.1.2    | Struttura del Proxy . . . . .                         | 30        |
| 4.1.3    | Struttura Vittima . . . . .                           | 31        |
| 4.2      | Implementazione del Timing Covert Channel . . . . .   | 32        |
| 4.2.1    | Randomizzazione dei tempi di invio . . . . .          | 36        |
| 4.2.2    | Requisiti per la comunicazione . . . . .              | 37        |
| 4.3      | Implementazione del Storage Covert Channel . . . . .  | 37        |
| 4.3.1    | Inserimento dei dati nei campi usati . . . . .        | 39        |
| 4.4      | Implementazione Behavioural Covert Channel . . . . .  | 48        |
| 4.5      | Implementazione di un Hybrid Covert Channel . . . . . | 51        |
| <b>5</b> | <b>Test e Risultati</b>                               | <b>53</b> |
| 5.1      | Test dei Covert Channel con RITA . . . . .            | 53        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 5.1.1    | Singolo invio di dati . . . . .                 | 54        |
| 5.1.2    | Molteplici invii dei dati . . . . .             | 55        |
| 5.1.3    | Considerazioni sui risultati ottenuti . . . . . | 62        |
| <b>6</b> | <b>Conclusioni</b>                              | <b>64</b> |

# Elenco delle figure

|      |                                                                       |    |
|------|-----------------------------------------------------------------------|----|
| 2.1  | Struttura pacchetto IPv4/ICMPv4 [14] . . . . .                        | 10 |
| 2.2  | Struttura del pacchetto <b>ICMPv4 Destination Unreachable</b> . . . . | 12 |
| 2.3  | Struttura del pacchetto <b>ICMPv6 Destination Unreachable</b> . . . . | 12 |
| 2.4  | Struttura del pacchetto <b>ICMPv4 Time Exceeded</b> . . . . .         | 13 |
| 2.5  | Struttura del pacchetto <b>ICMPv6 Time Exceeded</b> . . . . .         | 13 |
| 2.6  | Struttura del pacchetto <b>ICMPv4 Parameter Problem</b> . . . . .     | 13 |
| 2.7  | Struttura del pacchetto <b>ICMPv6 Parameter Problem</b> . . . . .     | 14 |
| 2.8  | Struttura del pacchetto <b>ICMPv4 Source Quench</b> . . . . .         | 14 |
| 2.9  | Struttura del pacchetto <b>ICMPv4 Redirect</b> . . . . .              | 15 |
| 2.10 | Struttura del pacchetto <b>ICMPv4 Echo</b> . . . . .                  | 16 |
| 2.11 | Struttura del pacchetto <b>ICMPv6 Echo</b> . . . . .                  | 16 |
| 2.12 | Struttura del pacchetto <b>ICMPv4 Timestamp</b> . . . . .             | 17 |
| 2.13 | Struttura del pacchetto <b>ICMPv4 Information</b> . . . . .           | 17 |
| 2.14 | Struttura del pacchetto <b>ICMPv6 Packet Too Big</b> . . . . .        | 18 |
| 4.1  | Architettura generale di <b>icmp tunnel</b> . . . . .                 | 25 |
| 4.2  | Comunicazione vitttima-proxy in <b>icmp tunnel</b> . . . . .          | 25 |
| 4.3  | Comunicazione proxy-internet in <b>icmp tunnel</b> . . . . .          | 26 |
| 4.4  | Flusso di comunicazione fra le entità . . . . .                       | 28 |
| 4.5  | Struttura e dialogo delle entità presenti . . . . .                   | 29 |
| 4.6  | Assegnazione degli intervalli con la distanza . . . . .               | 34 |
| 4.7  | Normalizzazione degli intervalli (Timing Channel) [20] . . . . .      | 36 |
| 4.8  | Randomizzazione dei tempi con il rumore (Timing Channel) . . . . .    | 37 |
| 4.9  | Messaggio IP/ICMP con campo <i>unused</i> azzerato . . . . .          | 39 |
| 4.10 | Messaggio IP/ICMP con campo <i>unused</i> impostato . . . . .         | 40 |

|      |                                                                      |    |
|------|----------------------------------------------------------------------|----|
| 4.11 | Messaggio IP/ICMP con campo <i>Header+64 bit</i> impostato . . . . . | 42 |
| 4.12 | Messaggio IP/ICMP con campo <i>pointer</i> impostato . . . . .       | 44 |
| 4.13 | Messaggio IP/ICMP con campo <i>identifier</i> impostato . . . . .    | 45 |
| 4.14 | Messaggio IP/ICMP con campo <i>Data</i> impostato . . . . .          | 46 |
| 4.15 | Messaggio IP/ICMP con campi <i>Timestamp</i> impostati . . . . .     | 47 |
| 4.16 | Schema Hybrid Channel [23] . . . . .                                 | 48 |
| 4.17 | Ricezione dei dati tramite Hybrid Covert Channel . . . . .           | 52 |

# Elenco delle tabelle

|     |                                                                                |    |
|-----|--------------------------------------------------------------------------------|----|
| 2.1 | Utilizzi del protocollo ICMP . . . . .                                         | 10 |
| 2.2 | Motivi del messaggio Destination Unreachable . . . . .                         | 11 |
| 2.3 | Motivi del messaggio Time Exceeded . . . . .                                   | 12 |
| 2.4 | Motivi del messaggio Redirect . . . . .                                        | 15 |
| 2.5 | Caratteristiche di un Covert Channel . . . . .                                 | 19 |
|     |                                                                                |    |
| 4.1 | Opzioni richieste nel comando dell'attaccante . . . . .                        | 30 |
| 4.2 | Opzioni richieste nel comando del proxy . . . . .                              | 31 |
| 4.3 | Opzioni richieste nel comando della vittima . . . . .                          | 32 |
| 4.4 | Parametri della prima funzione (Timing Channel) . . . . .                      | 34 |
| 4.5 | Struttura della tabella ASCII . . . . .                                        | 35 |
| 4.6 | Tipologie di messaggi ICMPv4 usati . . . . .                                   | 38 |
| 4.7 | Tipologie di messaggi ICMPv6 usati . . . . .                                   | 39 |
| 4.8 | Uso del Header+64 bits nel caso di IP/ICMP . . . . .                           | 41 |
| 4.9 | Struttura del campo Invoking Packet . . . . .                                  | 43 |
|     |                                                                                |    |
| 5.1 | Test tramite un singolo invio dei dati . . . . .                               | 55 |
| 5.2 | Test con payload fisso, 1MB di dati e un delay durante l'invio . . . . .       | 57 |
| 5.3 | Test con payload fisso, 1MB di dati e nessun delay durante l'invio . . . . .   | 59 |
| 5.4 | Test con payload fisso, 100KB di dati e un delay durante l'invio . . . . .     | 60 |
| 5.5 | Test con payload randomico, 100KB di dati e un delay durante l'invio . . . . . | 60 |
| 5.6 | Test con payload randomico, 1MB di dati e un delay durante l'invio . . . . .   | 62 |

# Capitolo 1

## Introduzione

La sicurezza informatica rappresenta oggi una delle sfide più critiche nell'ambito delle comunicazioni e delle reti informatiche. Data la continua evoluzione delle tecnologie e dato il costante aumento nella complessità dei sistemi informatici; emergono costantemente nuove vulnerabilità e tecniche di attacco che permetteranno l'esfiltrazione dei dati. Ciò ha richiesto un'attenzione particolare diventando un ambito primario di ricerca sia da parte di ricercatori universitari che di professionisti del settore. Tra queste tecniche, i covert channel costituiscono una minaccia particolarmente insidiosa, in quanto permettono la trasmissione di informazioni (fra due entità) in maniera non autorizzata sfruttando protocolli, canali di comunicazione e/o risorse legittimi. Ciò rendendo complessa la loro individuazione da parte dei tradizionali sistemi di sicurezza.

Il presente lavoro di tesi si concentra sull'analisi e sullo sviluppo di un covert channel basato sul protocollo ICMP (Internet Control Message Protocol) per l'estrazione di dati. Quest'ultimo risulta fondamentale per il funzionamento, il monitoraggio e la diagnostica delle reti IP e che, proprio per questo, non può essere completamente disabilitato o filtrato nelle reti moderne senza comprometterne l'operatività. Tutto questo lo rende un candidato ideale per la creazione di canali di comunicazione difficilmente rilevabili. L'obiettivo principale è dimostrare come il protocollo ICMP possa essere sfruttato per l'implementazione di diverse tipologie di Covert Channel che sono: i timing covert channel, gli storage covert channel ed i behavioural covert channel.

La tesi si articolerà in sei capitoli.

- Nel Capitolo 2 verranno inquadrati i Covert Channel ed il protocollo ICMP descrivendone i relativi concetti fondamentali; analizzando le diverse tipologie e caratteristiche dei canali nascosti sviluppabili e alle diverse tipologie di messaggi ICMP disponibili.
- Il Capitolo 3 descriverà gli strumenti software utilizzati per lo sviluppo del progetto ed il relativo testing. In esso sarà presente anche un'analisi di progetti esistenti che hanno già affrontato problematiche simili, identificandone punti di forza e criticità. Le osservazioni ricavate hanno poi guidato la progettazione dell'architettura descritta nel capitolo successivo.
- Il Capitolo 4 costituisce la parte centrale dell'elaborato: presenta l'implementazione dettagliata dell'architettura del sistema e della comunicazione fra le entità coinvolte (nello scambio delle informazioni). Descrive inoltre l'implementazione e lo sviluppo delle diverse tipologie di covert channel presenti. Di particolare importanza saranno le metodologie con cui si sono nascosti i dati sia attraverso i tempi dei messaggi sia attraverso i campi presenti in un pacchetto.
- I risultati dei test effettuati verranno riportati nel Capitolo 5. Ciò verrà fatto utilizzando RITA, come strumento per il rilevamento delle intrusioni, ed analizzando come diverse variazioni in alcuni parametri del covert channel influenzino la probabilità di essere individuati. In particolare i test potranno variare in base alla quantità di dati trasmessi, alla frequenza di invio dei dati, l'utilizzo dei mittenti reali o falsificati e la presenza (o la mancanza) di ritardi tra una trasmissione e la successiva.
- Infine, il Capitolo 6 presenta le conclusioni del lavoro e riassume i risultati ottenuti, evidenziando i fattori che influenzano maggiormente la rilevabilità di un covert channel. Vengono inoltre fornite raccomandazioni per il monitoraggio e limitazione del traffico ICMP nelle reti, al fine di mitigare i rischi associati a questo tipo di minaccia.



## Capitolo 2

# Covert Channel e Protocollo ICMP

### 2.1 Internet Control Message Protocol

ICMP[30] è un protocollo che opera al livello di rete (livello 3 nel modello ISO/OSI), e permette la **segnalazione di errori**, la **diagnostica della rete** e la **messaggistica di controllo**. Proprio per questo viene utilizzato per il monitoraggio dello stato di una rete e per la risoluzione dei problemi che avvengono in essa.

In un **Covert Channel ICMP** verranno utilizzati i messaggi ICMP per nascondere i dati. L'implementazione del canale è possibile siccome è un protocollo che, dati i suoi utilizzi (riassunti nella Tabella 2.1), non può essere del tutto disabilitato. Molti firewall e dispositivi di sicurezza consentono il traffico ICMP. Tuttavia, sebbene sia essenziale per la diagnostica di rete, può essere comunque utilizzato in modo improprio per degli attacchi o per la ricognizione della rete.

| Utilizzi               | Descrizione                                                                                        |
|------------------------|----------------------------------------------------------------------------------------------------|
| Diagnostica della rete | Il protocollo fornisce metriche critiche per il monitoraggio continuo delle prestazioni della rete |

|                             |                                                                                                                                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Segnalazione di errori      | Il protocollo rileva e segnala i problemi riscontrati durante la trasmissione dei dati tra i dispositivi sulla rete                                                  |
| Risoluzione dei problemi    | Gli amministratori di rete possono ricevere avvisi in tempo reale e rispondere rapidamente ai problemi di rete grazie agli strumenti di monitoraggio basati su ICMP. |
| Ottenimento di informazioni | Può inviare messaggi senza la necessità di una connessione preventiva permettendo così di ottenere informazioni.                                                     |

Tabella 2.1: Utilizzi del protocollo ICMP

I messaggi ICMP vengono inviati utilizzando l'intestazione IP di base. In essi i primi venti byte indicano l'intestazione IP mentre il primo ottetto, della porzione dati del datagramma, riguarda l'intestazione ICMP. Nell'intestazione, nel caso del protocollo ICMP, il campo *protocol* avrà valore 1.

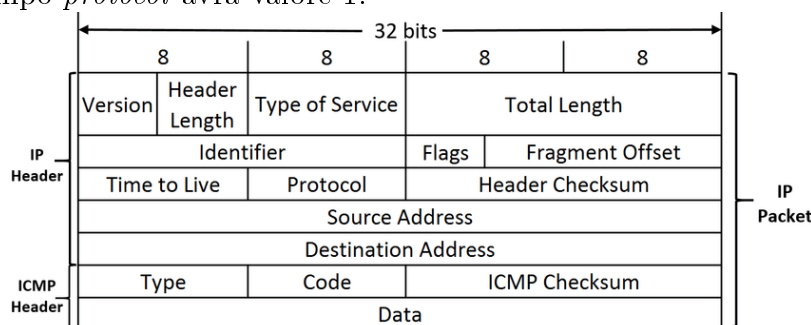


Figura 2.1: Struttura pacchetto IPv4/ICMPv4 [14]

- **Type:** Identifica la tipologia di messaggio. In base a questo campo verrà determinato il formato dei rimanenti dati.
- **Code:** Fornisce dettagli aggiuntivi sulla tipologia di messaggio.

- **Checksum:** Garantisce l'integrità dei dati.
- **Data:** Campo opzionale, può contenere ulteriori dati.

### 2.1.1 Tipologie di Messaggi ICMP

In base alle linee guida definite nella RFC 792 (ICMPv4)[6] e nella RFC 4443 (ICMPv6)[5], I messaggi presenti in ICMP sono classificati o come messaggi di errore o come messaggi informativi. I primi segnalano problemi nella comunicazione di rete mentre i secondi vengono utilizzati per scopi diagnostici e di controllo.

#### Destination Unreachable

Viene inviato quando il pacchetto non può essere recapitato alla destinazione specificata nell'header IP. Di solito perchè il percorso definito non può essere seguito. Il messaggio non verrà (e non dovrà essere) generato se un pacchetto viene scartato a causa della congestione del traffico. Il codice impostato nel messaggio indicherà il motivo dell'invio; i quali sono indicati nella Tabella 2.2.

| Codice                    | Descrizione                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------------|
| Rete irraggiungibile      | La rete di destinazione non è raggiungibile.                                                           |
| Host irraggiungibile      | Il router può raggiungere la rete ma non l'host specifico. siccome non risponde o non è raggiungibile. |
| Protocollo non attivo     | Il protocollo specificato nel pacchetto IP non è attivo.                                               |
| Porta non attiva          | La porta di destinazione non è attiva o nessun servizio è in ascolto.                                  |
| Necessaria frammentazione | È necessaria la frammentazione del pacchetto ma il flag "Don't Fragment" (DF) è impostato.             |

Tabella 2.2: Motivi del messaggio Destination Unreachable

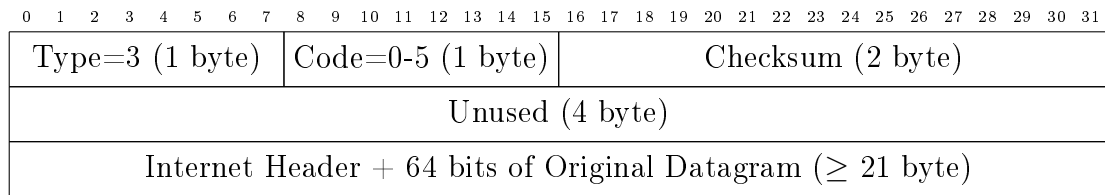


Figura 2.2: Struttura del pacchetto **ICMPv4 Destination Unreachable**

Il campo *Internet Header*: viene utilizzato dall'host per accoppiare il messaggio di errore al processo appropriato.

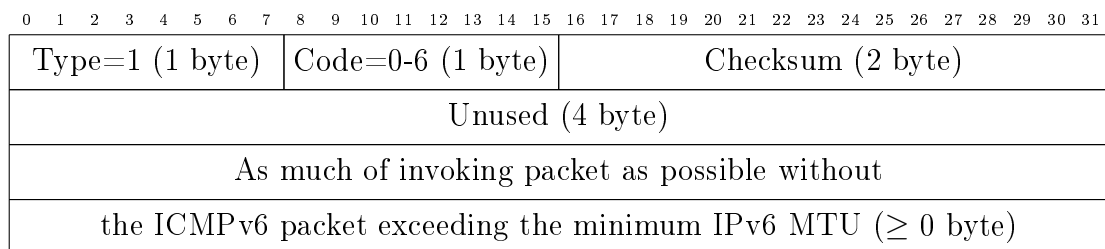


Figura 2.3: Struttura del pacchetto **ICMPv6 Destination Unreachable**

Il campo *Invoking Packet* indica quanta parte del pacchetto (che ha attivato l'errore ICMPv6) debba essere inclusa. Il tutto senza eccedere il *IPv6 MTU* il cui valore di default equivale a 1280 bytes.

## Time Exceeded

Questa tipologia di messaggio viene usata quando il gateway che elabora un pacchetto trova che il suo TTL (tempo di vita) è zero. In questi casi il gateway dovrà scartare il datagramma e notificare l'host sorgente della cosa. Il codice impostato nel messaggio indicherà uno dei motivi indicati nella Tabella 2.3.

| Evento                                  | Esempio                                                                          |
|-----------------------------------------|----------------------------------------------------------------------------------|
| TTL scaduto                             | Il TTL specificato inizialmente è troppo basso o è presente un loop nel routing. |
| Tempo per il riasset-<br>taggio scaduto | Il tempo per riassemble i frammenti scaduto.                                     |

Tabella 2.3: Motivi del messaggio Time Exceeded

|                                                                  |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------------------------------------------------------------|---|---|---|---|---|---|---|-------------------|---|----|----|----|----|----|----|-------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                                                                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8                 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=11 (1 byte)                                                 |   |   |   |   |   |   |   | Code=0-1 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Unused (4 byte)                                                  |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Internet Header + 64 bits of Original Datagram ( $\geq 21$ byte) |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.4: Struttura del pacchetto **ICMPv4 Time Exceeded**

In ICMPv4 il campo *Internet Header*: viene utilizzato dall'host per accoppiare il messaggio di errore al processo appropriato.

|                                                                   |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------------------------------------------------------------|---|---|---|---|---|---|---|-------------------|---|----|----|----|----|----|----|-------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                                                                 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8                 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=3 (1 byte)                                                   |   |   |   |   |   |   |   | Code=0-1 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Unused (4 byte)                                                   |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| As much of invoking packet as possible without                    |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| the ICMPv6 packet exceeding the minimum IPv6 MTU ( $\geq 0$ byte) |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.5: Struttura del pacchetto **ICMPv6 Time Exceeded**

In ICMPv6 il campo *Invoking Packet* indica quanta parte del pacchetto (che ha attivato l'errore ICMPv6) debba essere inclusa. Il tutto senza eccedere il *IPv6 MTU* il cui valore di default equivale a 1280 bytes.

## Parameter Problem

Viene usata quando il gateway che elabora un pacchetto trova un problema con i parametri dell'intestazione in modo tale da non poter completare l'elaborazione del datagramma. In questo caso dovrà scartare il datagramma e notificare la cosa all'host indicando il tipo e la posizione del problema. Il messaggio viene inviato solo se l'errore ha causato lo scarto del pacchetto.

|                                                                  |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------------------------------------------------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|-------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                                                                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=12 (1 byte)                                                 |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Pointer (1 byte)                                                 |   |   |   |   |   |   |   | Unused (3 byte) |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Internet Header + 64 bits of Original Datagram ( $\geq 21$ byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.6: Struttura del pacchetto **ICMPv4 Parameter Problem**

In ICMPv4 il campo *Internet Header*: viene utilizzato dall'host per accoppiare il messaggio di errore al processo appropriato.

Il puntatore identifica l'ottetto nell'intestazione del pacchetto originale in cui è stato rilevato l'errore.

|                                                                   |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------------------------------------------------------------|---|---|---|---|---|---|---|-------------------|---|----|----|----|----|----|----|-------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                                                                 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8                 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=4 (1 byte)                                                   |   |   |   |   |   |   |   | Code=0-2 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Pointer (4 byte)                                                  |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| As much of invoking packet as possible without                    |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| the ICMPv6 packet exceeding the minimum IPv6 MTU ( $\geq 0$ byte) |   |   |   |   |   |   |   |                   |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.7: Struttura del pacchetto **ICMPv6 Parameter Problem**

In ICMPv6 il campo *Invoking Packet* indica quanta parte del pacchetto (che ha attivato l'errore ICMPv6) debba essere inclusa. Il tutto senza eccedere il *IPv6 MTU* il cui valore di default equivale a 1280 bytes.

Il puntatore identifica l'ottetto nell'intestazione del pacchetto originale in cui è stato rilevato l'errore.

## Source Quench

Questa tipologia viene usata quando il gateway vuole richiedere di ridurre la velocità di invio dei pacchetti. Questo perchè il gateway ha scartato un pacchetto ma non a causa di un errore. Al suo ricevimento, l'host sorgente dovrà ridurre la velocità sino a quando non riceverà più messaggi del tipo Source Quench dal gateway.

|                                                                  |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------------------------------------------------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|-------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                                                                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=4 (1 byte)                                                  |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Unused (4 byte)                                                  |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Internet Header + 64 bits of Original Datagram ( $\geq 21$ byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.8: Struttura del pacchetto **ICMPv4 Source Quench**

Il campo *Internet Header*: viene utilizzato dall'host per accoppiare il messaggio di errore al processo appropriato. Se un protocollo di livello superiore utilizza numeri di porta, si presume che siano nei primi 64 bit dei dati del datagramma originale.

## Redirect

Indica un messaggio di reindirizzamento a un host. Il gateway manda questo tipo di messaggio se, dopo aver controllato la sua tabella di routing, trova che esiste un gateway migliore che si trova sulla sua stessa rete. Questo secondo gateway rappresenterà un percorso migliore per la destinazione. Il codice impostato nel messaggio indicherà uno dei motivi presenti nella Tabella 2.4.

| Evento         | Esempio                                                                                                              |
|----------------|----------------------------------------------------------------------------------------------------------------------|
| Route migliore | Il gateway riceve il pacchetto e dalla tabella di routing ottiene che il secondo gateway si trova sulla stessa rete. |

Tabella 2.4: Motivi del messaggio Redirect

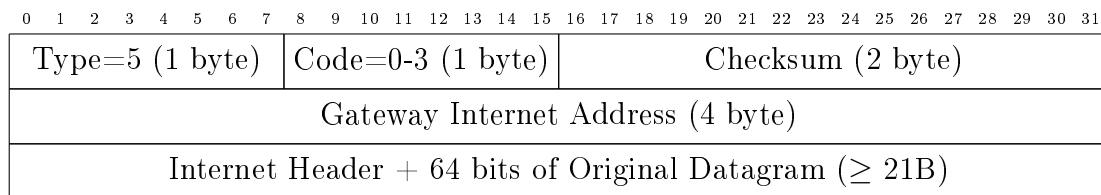


Figura 2.9: Struttura del pacchetto **ICMPv4 Redirect**

Nel campo *Gateway Internet Address* verrà indicato l'indirizzo del nuovo gateway a cui dovrà essere inviato il traffico per la rete di destinazione (specificata nel campo di destinazione del datagram originale). Si potrebbe pensare di utilizzare il campo ma un gateway o la vittima per necessità potrebbero leggere i dati e scoprire che non sono conformi.

Il campo *Internet Header* viene utilizzato dall'host per accoppiare il messaggio di errore al processo appropriato. Se un protocollo di livello superiore utilizza numeri di porta, si presume che siano nei primi 64 bit dei dati del datagramma originale.

Se nell'itestazione IP è presente l'opzione *IP Source Route*, il messaggio di reindirizzamento non verrà inviato anche se è presente un percorso migliore.

## Echo Request / Echo Reply

Un messaggio *Echo*, viene usato per ricevere indietro una risposta da un host. Si inviano dei dati tramite una Echo Request, e questi'ultimi dovranno essere restituiti in un messaggio di risposta integralmente e senza modifiche.

|                           |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |
|---------------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|--------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|--|--|--|--|--|--|--|--|
| 0                         | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                       | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |  |  |  |  |  |  |  |  |
| Type=8 (1 byte)           |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte)        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |
| Identifier (2 byte)       |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    | Sequence Number (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |
| Data ... ( $\geq 0$ byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |

Figura 2.10: Struttura del pacchetto **ICMPv4 Echo**

|                     |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |
|---------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|--------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|--|--|--|--|--|--|--|--|
| 0                   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                       | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |  |  |  |  |  |  |  |  |
| Type=128 (1 byte)   |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte)        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |
| Identifier (2 byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    | Sequence Number (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |
| Data ... (≥ 0B)     |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |  |  |  |  |  |  |  |

Figura 2.11: Struttura del pacchetto **ICMPv6 Echo**

Nel messaggio, i campi identificatore e numero di sequenza possono essere utilizzati dal mittente per facilitare l'abbinamento delle risposte con le richieste.

Il mittente non ha alcuna limitazione sulla quantità di dati inseribili nel campo del payload. Tuttavia una limitazione potrà essere data dalla massima capacità di trasporto dei collegamenti. Nel caso la dimensione del messaggio la superasse; il pacchetto dovrà essere frammentato per poter essere spedito. In media il valore si attesta sui 1400 bytes [8].

## Timestamp Request / Timestamp Reply

Viene usato per ricevere indietro una risposta da un host. I dati ricevuti nel messaggio di richiesta, vengono restituiti in quello di risposta insieme a dei timestamp aggiuntivi. Il timestamp è pari a 32 bit e indica i millisecondi che sono passati dalla mezzanotte UT.



|                              |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------------------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|--------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                            | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                       | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=13 (1 byte)             |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte)        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Identifier (2 byte)          |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    | Sequence Number (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Originate Timestamp (4 byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Receive Timestamp (4 byte)   |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Transmit Timestamp (4 byte)  |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Data ... ( $\geq 0$ byte)    |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.12: Struttura del pacchetto **ICMPv4 Timestamp**

L'identificatore e il numero di sequenza possono essere utilizzati dal mittente del pacchetto per facilitare l'abbinamento delle risposte con le richieste. Mentre i campi relativi ai timestamp indicheranno rispettivamente: il tempo in cui il mittente ha toccato il messaggio per l'ultima volta prima di inviarlo, il tempo in cui il destinatario ha toccato per la prima volta il messaggio (alla ricezione) e il tempo in cui il destinatario ha toccato il messaggio per l'ultima volta prima di inviarlo.

### Information Request / Information Reply

La tipologia *Information* viene usata per consentire di scoprire il numero della rete in cui un host si trova. Serve quindi per capire se si trova nella stesse rete dell'host che risponde. Sebbene il messaggio può essere inviato con la destinazione nell'intestazione IP pari a zero (ciò significa "questa" rete); l'intestazione IP presente nel messaggio di risposta dovrà essere inviata con gli indirizzi IP completamente specificati.

|                     |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                          |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|--------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                       | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=15 (1 byte)    |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte)        |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| Identifier (2 byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    | Sequence Number (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.13: Struttura del pacchetto **ICMPv4 Information**

L'identificatore e il numero di sequenza possono essere utilizzati dal mittente del pacchetto per facilitare l'abbinamento delle risposte con le richieste.

## Packet Too Big

Un messaggio *Packet Too Big* viene generato da un router in risposta a un pacchetto che non può inoltrare perché è più grande dell'MTU del collegamento in uscita. Un nodo che riceve un messaggio *ICMPv6 Packet Too Big* deve notificare la cosa al processo di livello superiore (se il processo in questione può essere identificato).

|                                                                   |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-------------------------------------------------------------------|---|---|---|---|---|---|---|-----------------|---|----|----|----|----|----|----|-------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0                                                                 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8               | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16                | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| Type=2 (1 byte)                                                   |   |   |   |   |   |   |   | Code=0 (1 byte) |   |    |    |    |    |    |    | Checksum (2 byte) |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| MTU (4 byte)                                                      |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| As much of invoking packet as possible without                    |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| the ICMPv6 packet exceeding the minimum IPv6 MTU ( $\geq 0$ byte) |   |   |   |   |   |   |   |                 |   |    |    |    |    |    |    |                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Figura 2.14: Struttura del pacchetto **ICMPv6 Packet Too Big**

Il campo *MTU* indica la massima unità di trasmissione del collegamento nel salto successivo. Mentre il campo *Invoking Packet* indica quanta parte del pacchetto (che ha attivato l'errore ICMPv6) debba essere inclusa. Il tutto senza eccedere il *IPv6 MTU* il cui valore di default equivale a 1280 bytes.

## 2.2 Covert Channel

Un **Covert Channel**[15][7] è un attacco che permette (in ambienti ritenuti sicuri) la capacità di comunicare e/o trasferire dati in maniera non autorizzata e non voluta. Solitamente operano al di fuori degli usuali meccanismi di comunicazioni sfruttando vulnerabilità o comportamenti non previsti nei sistemi. Ciò gli permette di non generare segnali di un uso improprio del sistema ed inoltre, nascondendosi all'interno dei normali processi del sistema, sono difficili da rilevare e/o identificare.

Qualsiasi risorsa condivisa può essere utilizzata per la creazione di un canale nascosto. E per poter essere efficace, un Covert Channel deve possedere determinate caratteristiche. Fra quelle indicate nella Tabella 2.5; l'**indistinguibilità** è quella principale.

È estremamente importante riuscire a trasmettere informazioni mantenendo conforme lo stato del sistema. L'obiettivo è rendere il canale indistinguibile rispetto alle altre risorse presenti nel sistema così da risultare invisibili ai sistemi di monitoraggio.

| <b>Caratteristica</b>           | <b>Descrizione</b>                                                                                                                                                             |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Furtività</b>                | Evitare di attirare le attenzioni sia degli amministratori che degli strumenti utilizzati per il rilevamento degli attacchi.                                                   |
| <b>Capacità di trasmissione</b> | Più dati il canale trasmette per un determinato intervallo di tempo, maggiore sarà il rischio che venga scoperto.                                                              |
| <b>Uso delle risorse</b>        | Un uso delle risorse improprio o sproporzionato da parte del canale potrebbe andare in conflitto con le risorse legittime presenti nel sistema.                                |
| <b>Rumore</b>                   | L'uso dei servizi e/o delle risorse nel sistema potrebbe alterare il loro funzionamento. Ciò potrebbe attirare l'attenzione da parte degli amministratori.                     |
| <b>Indistinguibilità</b>        | La trasmissione dei dati mantiene conforme e inalterato il funzionamento delle risorse utilizzate. L'obiettivo è quello di rendersi indistinguibili dalla risorsa autorizzata. |

Tabella 2.5: Caratteristiche di un Covert Channel

## **2.2.1 Tipologie di Covert Channel**

### **Timing Covert Channel**

I covert channel di temporizzazione sono metodi di comunicazione che permettono ad un osservatore (un umano o un processo) di acquisire informazioni attraverso il cambiamento del tempo di risposta di una risorsa. Qualsiasi metodo che utilizza un orologio (o una misurazione del tempo) per segnalare il valore può implementarlo.

### **Storage Covert Channel**

Il canale viene creato scrivendo dei dati su un'area di memoria condivisa accessibile da tutte entità presenti. I possibili veicoli saranno tutte quelle risorse che consentiranno la scrittura (diretta o indiretta) da parte di un processo e la lettura (diretta o indiretta) da parte di un altro.

### **Behavioural Covert Channel**

Le informazioni vengono codificate modificando il comportamento del sistema. Quindi i dati vengono ricavati osservando il comportamento del sistema piuttosto che attraverso l'accesso diretto ai dati.

### **Hybrid Covert Channel**

Un Hybrid Covert Channel combina più tecniche per aumentare la capacità di trasmissione dei dati nascosti e rendere più difficile la loro rilevazione.

# Capitolo 3

## Strumenti Utilizzati

Il presente capitolo descrive gli strumenti software ed i framework adottati per lo svolgimento delle attività di sviluppo e per la creazione degli ambienti di test che verranno successivamente nella tesi. L

Il capitolo si articolerà in diverse sezione; ciascuna delle quali viene dedicata a un componente specifico dell'infrastruttura. L'ambiente operativo si compone di una piattaforma di virtualizzazione per la creazione di macchine virtuali<sup>3</sup>, di un framework per la manipolazione e l'analisi dei pacchetti di rete<sup>3</sup>, e di strumenti per il rilevamento di potenziali minacce informatiche<sup>3</sup>. Particolare attenzione sarà anche dedicata all'analisi di progetti esistenti <sup>3 3 3</sup> che hanno già affrontato la tematica dei covert channel tramite ICMP. L'analisi di questi progetti ha permesso di identificare i requisiti fondamentali, i punti di forza e le criticità delle diverse implementazioni, fornendo una base solida per la progettazione delle soluzione proposte.

Lo scopo di questa sezione sarà quindi quello di fornire una panoramica delle risorse utilizzate; così da porre le basi per la comprensione delle successive fasi di analisi, implementazione, sperimentazione e valutazione dei risultati.

### **Virtual Box**

VirtualBox [21] è un software di virtualizzazione open source che consente di eseguire più sistemi operativi (guest) su un singolo computer (host).

Il suo utilizzo è stato necessario per poter testare il codice sviluppato in un ambiente Linux. Sono state quindi create, per ciascun entità necessaria, una macchina virtuale contenente Ubuntu.

Alla fine si sono ottenute quattro macchine virtuali: una per l'attaccante e la vittima, mentre le altre due per i proxy. Si poteva usare anche un singolo proxy ma si voleva testare come i dati ricavati dalla vittima (e da inoltrare all'attaccante), venissero distribuiti ai proxy disponibili.

## Scapy

Scapy [24] è un framework per la manipolazione dei pacchetti scritto in Python che consente di falsificare varie tipologie di pacchetti (http, tcp, ip, udp, icmp, ecc.) È in grado di creare o decodificare pacchetti di vari protocolli; può inviarli in rete, catturarli, memorizzarli o leggerne i dati.

Svolge principalmente due funzioni: invia i pacchetti e riceve le risposte, consentendo all'utente di inviare, intercettare, analizzare e falsificare pacchetti di rete. Questa capacità ha consentito la creazione di uno strumento in grado di ricevere o inviare pacchetti ICMP modificati per nascondere dei dati.

Nella libreria il metodo **send** (o similari) permetteranno di inviare un definito pacchetto. Invece per definire un pacchetto si concatenano i livelli [25] indicanti i protocolli presenti (nel nostro caso IP e ICMP). Per poter ascoltare il traffico di rete [26] è presente il metodo **sniff** oppure si può definire una variabile di tipo *AsyncSniffer*; che permette un grado di libertà maggiore siccome ha dei metodi che se chiamati terminano lo sniffing.

## RITA (Real Intelligence Threat Analytics)

RITA [2] è uno strumento open source per la ricerca delle minacce di rete, progettato per identificare attività di comando e controllo (C2) dannose. Acquisisce i log di Zeek e utilizza l'analisi comportamentale per identificare sistemi potenzialmente compromessi. Per l'installazione si è seguita la guida ufficiale messa a disposizione da Active Countermeasures<sup>1</sup>. Siccome si utilizza un computer Windows, i comandi sono stati

---

<sup>1</sup><https://www.activecountermeasures.com/free-tools/rita/>

esguiti tramite WSL (Windows Subsystem for Linux).

Le funzionalità principali sono: il rilevamento dei Beacon, rilevamento del tunneling DNS, tempi di durata delle connessioni, controllo dei feed per le Threat Intel (domini e host sospetti), valutazione per gravità delle connessioni, quanti host hanno comunicato con un determinato host, il primo incontro di un host,

Una costante assoluta su cui RITA fa affidamento per rilevare i malware, è che dovranno chiamare "casa". Da questo presupposto; analizzando il traffico di rete, rilevare le chiamate C2 indipendentemente dalla piattaforma.

La persistenza è l'attributo chiave che si ricerca quando si analizza sistemi compromessi. RITA cerca gli indicatori principali relativi a questa persistenza. Viene fatto acquisendo i log di connessione di Zeek e tramite l'utilizzo dell'analisi comportamentale identifica i sistemi potenzialmente compromessi.

## ICMP Door

Il programma [16] è stato studiato per comprendere come potesse effettuare il tunneling dei dati. Oltre alla struttura delle entità, che successivamente verrà ridefinita, risulta interessante come il programma richiede degli argomenti dall'utente. Tramite la libreria *argparse* richiede all'utente l'interfaccia su cui ascoltare i dati e l'indirizzo di destinazione dei pacchetti. Inoltre usa il metodo *sniff* del framework Scapy per ascoltare il flusso dei dati mentre tramite *sr* invia i pacchetti.

Sebbe il programma ci introduce a una possibile struttura del Covert Channel; gli si sono trovati dei difetti. Gli svantaggi sono che i dati vengono trasmessi non solo nel campo data, del messaggio di tipologia ICMP Echo Reply, ma anche in chiaro. Inoltre il valore del campo identifier rimano invariato per tutta la sessione. Un sistema di sicurezza, se vedesse le molteplici risposte (che non combaciano con il numero di richieste) e leggesse i testi in chiaro, potrebbe identificare il canale nascosto. Di solito per ogni Echo Request corrisponde una singola Echo Reply in cui la risposta rimanda i dati ricevuti e il campo data di solito contiene frasi già preimpostate e sempre costanti (e.g. 'helloworld').

## ICMP Exfil

Si è analizzato ICMP Exfil [17] per vedere come un Covert Channel temporalizzato potesse funzionare. Riceve un dato, lo converte in binario e dopodichè avrà una lista di numeri binari. Per mandare il dato, invia un ping con un timeout pari a **binary\_number+leway**. Il valore leway viene utilizzato per rallentare il numero di pacchetti inviati ed avere una connessione maggiormente silenziosa. L'autore infine indica come migliorare, la crittografia dei dati per aggiungere del rumore, dell'entropia.

Ciò su cui si affida è che un osservatore, vedendo i pacchetti ICMP, li veda come validi; Siccome non riuscirebbero a trovare alcun dato, a meno che non sappiano della tecnica utilizzata.

## ICMP Tunnel

È uno strumento [3] che permette il tunneling del traffico IP. Tramite delle richieste e risposte ICMP Echo, incapsula il traffico e lo invia al server proxy. Quest'ultimo lo decapsula e lo inoltra. I pacchetti in entrata, che sarebbero diretti alla macchina vittima, sono poi incapsulati dal proxy e inviati. L'approccio è possibile siccome RFC-792, che indica le linee guida del protocollo ICMP, permette una quantità arbitraria di dati nei pacchetti ICMP Echo (sia richiesta che risposta).



# Capitolo 4

## Implementazione

Dopo aver analizzato gli strumenti che già hanno provato a sviluppare un covert channel tramite ICMP e dopo aver analizzato metodologie sfruttabili per poter nascondere i dati; procediamo nel svilupparne uno; il cui codice è contenuto in una repository GitHub[18].

Per la definizione della struttura, si è preso spunto da **icmp tunnel** [3]; il cui schema generale di funzionamento viene mostrato nella Figura 4.1.

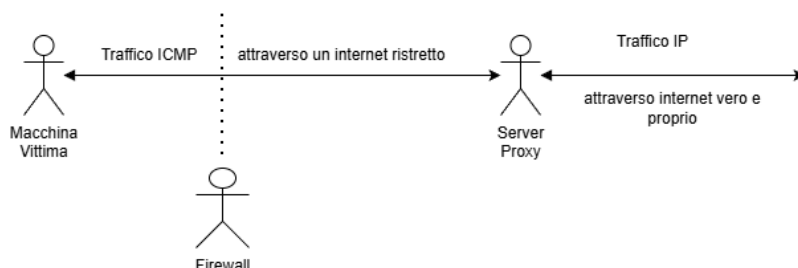


Figura 4.1: Architettura generale di **icmp tunnel**

La Figura 4.2 mostra la struttura della comunicazione fra la macchina vittima e il proxy. *Icmp tunnel* che è in esecuzione sulla macchina vittima, reindirizza il traffico IP ad un interfaccia virtuale per poi inoltrarlo al proxy tramite il protocollo ICMP.

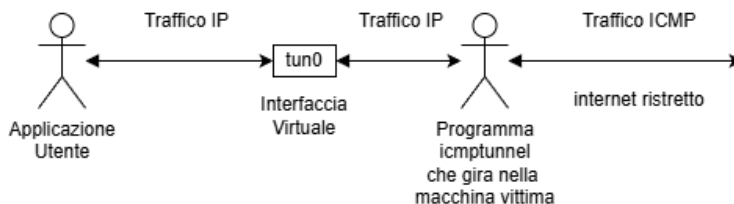


Figura 4.2: Comunicazione vittima-proxy in **icmp tunnel**

Il proxy intanto rimane in ascolto di questi pacchetti ICMP e poi li invia, utilizzando il protocollo IP, verso la destinazione finale. Successivamente i pacchetti in entrata nel proxy, che contengono le informazioni da inoltrare alla macchina vittima, vengono incapsulati tramite il protocollo ICMP ed inoltrati alla vittima. La Figura 4.3 mostra la struttura di quest'ultima parte.

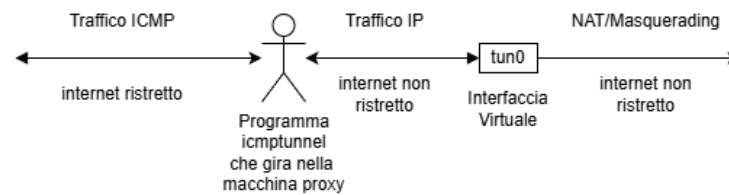


Figura 4.3: Comunicazione proxy-internet in **icmp tunnel**

Alcuni aspetti da considerare, utili l'implementazione della comunicazione, sono:

1. La **presenza di più proxy intermediari** fra l'attaccante e la vittima. Sebbene sia possibile usare un singolo proxy, l'utilizzo di più macchine proxy permette di nascondere meglio la comunicazione.

Con un solo proxy, il sistema di difesa potrebbe notare che tutto il traffico ICMP viene inviato ad un'unica sorgente. Se invece si utilizzano più proxy, il traffico può essere distribuito fra più destinatari diversi.

2. La **quantità di dati** che si vuole inviare. Se mai si volesse esfiltrare un file; la grande quantità di dati potrebbe generare rumore.

Si deve tenere conto di un **limite massimo** di dati inviabili in un intervallo di tempo ed un **periodo di riposo** che deve trascorrere prima di poterne inviare altri.

3. La **trasmissione dei dati in chiaro** permetterebbe a chiunque di leggerne il contenuto. Un sistema di difesa che fa uso dell'ispezione approfondita dei pacchetti (Deep Packet Inspection) potrebbe rilevare la comunicazione. Tuttavia anche la **trasmissione di dati cifrati** potrebbe destare sospetti.

Si devono quindi poter mandare dati in maniera cifrata ma che al tempo stesso

non sembrano cifrati. Al momento non si è trovato un metodo efficace a parte l'inserimento di dati in forma numerica (anche se non viene considerata una cifratura ma più una decodifica del carattere <sup>1</sup>).

4. I **messaggi di tipo Request**, inviano un messaggio di richiesta e poi aspettano una risposta avente gli stessi dati ricevuti. Si avranno quindi due messaggi identici (tranne per il campo che indica il tipo di messaggio). Ciò risulta un problema se la dimensione del campo contenente i dati è eccessiva. Inoltre il numero di messaggi che inviati raddoppierà.

Delle possibili soluzioni sono:

- l'uso dei solo messaggi di risposta (di tipo **Reply**). Tuttavia sarà anomalo che il numero delle risposte non combaci con quello delle richieste.
- la disabilitazione, se possibile, dell'invio da parte del sistema delle risposte così da mandare una risposta specifica (che non ripeterà il contenuto della richiesta). In questo caso ad una richiesta combaccerà una risposta sebbene non conforme agli standard.

## 4.1 Struttura della comunicazione fra le entità

La Figura 4.4 mostra il flusso di comunicazione fra le entità coinvolte nella comunicazione covert channel. Di seguito la descrizione di ciascuna entità.

- **Attaccante:**

Carica un file di configurazione nel quale viene definito l'indirizzo IP della vittima, il metodo di attacco e i proxy utilizzabili per farlo. Inoltre inserirà il comando che la vittima dovrà eseguire o il dato che gli si vuole mandare. Nel caso utilizzasse dei proxy, aspetterà da essi i dati che la vittima restituirà.

- **Proxy:**

Ha bisogno di sapere l'indirizzo IP dell'attaccante siccome deve ottenere da esso l'indirizzo IP della vittima e il comando da inoltrarle. Una volta stabilita una

---

<sup>1</sup>Questo perchè un carattere ASCII può essere codificato tramite un numero intero.

connesione sia con l'attaccante che con la vittima; si occuperà di inoltrare i dati che le due entità si scambiano.

- **Vittima:**

Aspetta le connessioni o dall'attaccante o dai proxy (se vengono utilizzati). Rimane in attesa di un comando (o di un dato). Nel primo caso lo eseguirà e successivamnete invierà i dati ricavati.

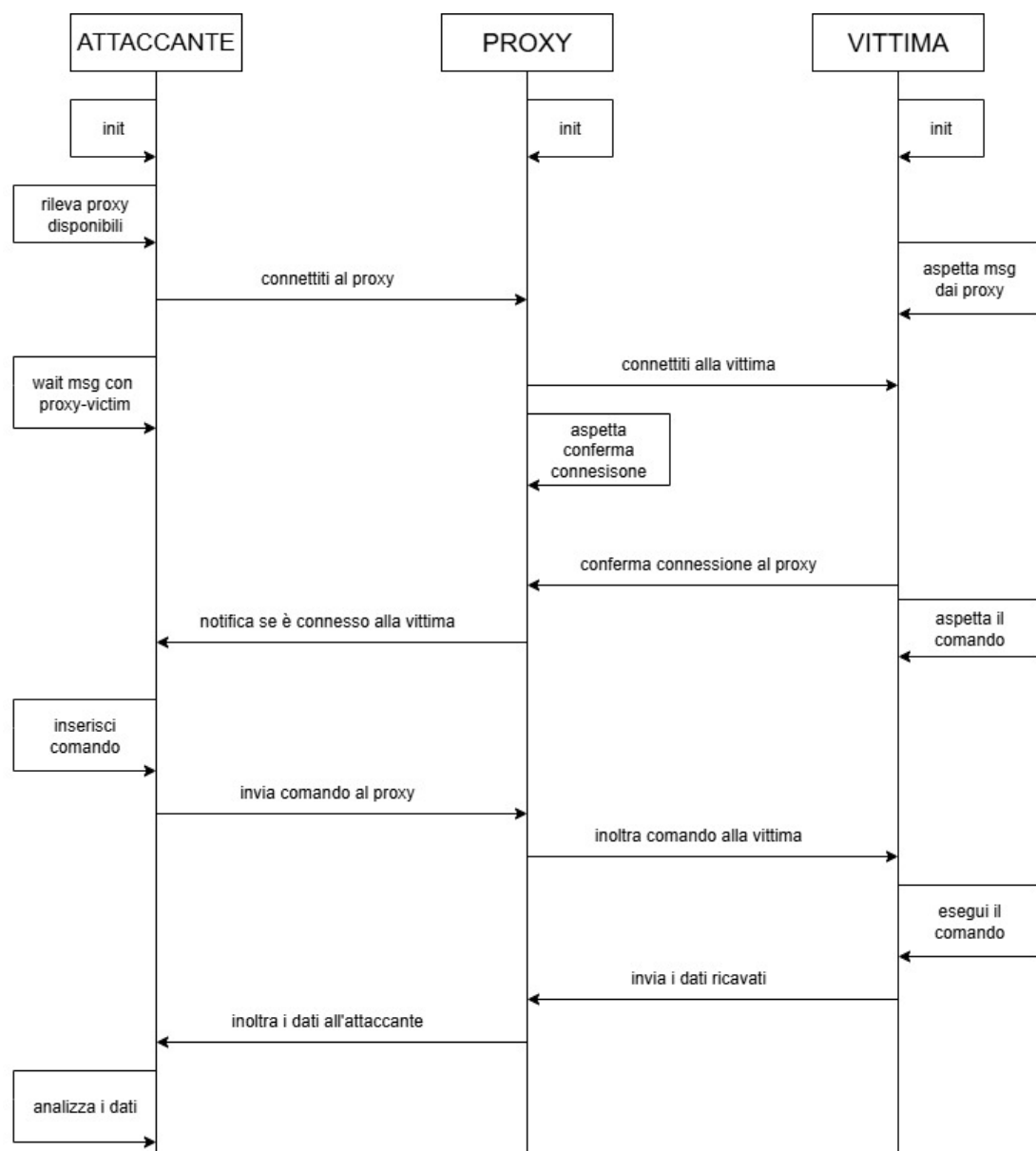


Figura 4.4: Flusso di comunicazione fra le entità

Il canale di comunicazione che il proxy e l'attaccante potranno avere dipende dall'affidabilità di quest'ultimo<sup>2</sup>:

- Nel caso si reputi il proxy insicuro, si potrà utilizzare una comunicazione TCP così da avere una comunicazione stabile e affidabile.
- Se invece si reputa il proxy non sicuro (e quindi si dovrà nascondere la comunicazione anche da lui), si potrà utilizzare una comunicazione usata con la vittima. Quindi il proxy e l'attaccante comunicheranno tramite pacchetti ICMP. Questa tipologia di comunicazione è più instabile rispetto alla precedente ma garantisce un maggior anonimato.

Riguardo la comunicazione; l'attaccante potrà usare uno (il caso standard) o più proxy per comunicare con la vittima (come mostrato nella Figura 4.5). Tuttavia sarà anche possibile (per l'attaccante) anche comunicare direttamente con la vittima. In questo caso alcune funzioni presenti nel proxy verranno eseguite dall'attaccante (e.g. connettersi alla vittima). Mentre nel precedente invece, si deve prevedere un metodo per poter riordinare i messaggi che l'attaccante ha ricevuto dai proxy<sup>3</sup>.

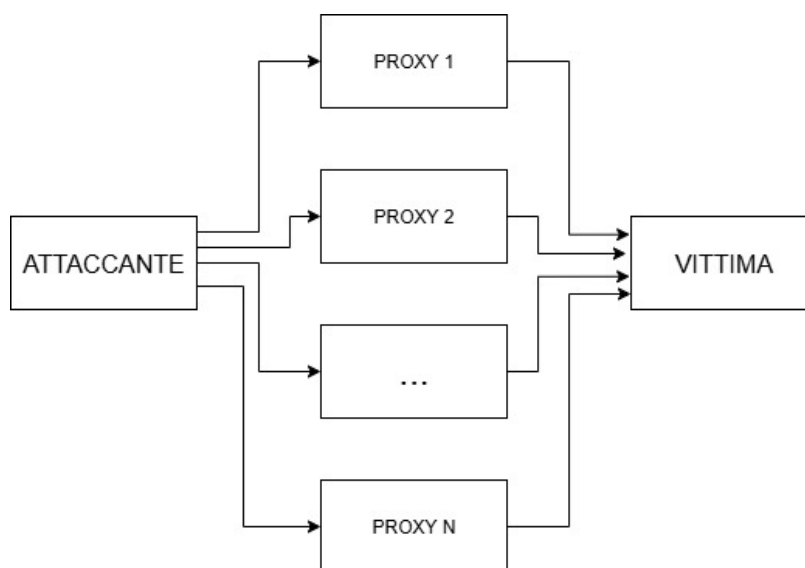


Figura 4.5: Struttura e dialogo delle entità presenti

<sup>2</sup>Durante lo sviluppo si sono ritenuti i proxy affidabili

<sup>3</sup>Ciò è stato fatto inserendo un indice al pacchetto inviato dal proxy

### 4.1.1 Struttura dell'attaccante

Tramite un parser rileve se nella linea di comando sono state inserite le opzioni descritte nella Tabella 4.1. Queste sono necessarie alla sua inizializzazione.

| Opzione   | Utilizzo                                                                                                                                                                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Path file | Percorso per il file di configurazione da caricare. In questo file JSON viene specificato l'indirizzo IP della vittima, la metodologia di attacco e la lista dei proxy che si vogliono usare. |

Tabella 4.1: Opzioni richieste nel comando dell'attaccante

Per ogni proxy specificato nel file di configurazione, si verifica quali sono disponibili; ovvero quali proxy sono connessi sia all'attaccante che alla vittima. Per farlo si crea un canale di comunicazione con ciascun proxy, o tramite ICMP o TCP/IP, e si invia un messaggio di connessione. In questo messaggio si specifica sia l'indirizzo IP della vittima che la metodologia di attacco scelta. Successivamente si rimarrà in attesa che il proxy confermi la connessione con la vittima.

Nel caso non si usassero dei sockets TCP/IP ma il protocollo ICMP, bisognerà definire un thread che si occupi di monitorare il traffico di rete per poter catturare i messaggi ICMP contenenti i dati inviati dai proxy. Quindi prima di inviare alcun comando (o dato), bisogna assicurarsi che il thread sia già partito altrimenti si potrebbero perdere delle informazioni.

Una volta ricevuti tutti i dati inoltrati dai proxy, l'attaccante dovrà riordinarli per ricavare il messaggio. Inoltre se si volesse mandare un altro comando, si dovranno reimpostare le variabili necessarie per farlo. Altrimenti si può decidere di interrompere la comunicazione e in quel caso i proxy ne verranno aggiornati.

### 4.1.2 Struttura del Proxy

Come per l'attaccante, anche il proxy necessita di alcune opzioni (descritte nella Tabella 4.2).

| Opzione      | Utilizzo                                                                                                                                                                                                                                                                               |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Indirizzo IP | Rappresenta l'indirizzo IP dell'attaccante. Quando il proxy crea il socket e rimane in ascolto delle connessioni, accetterà solo quelle che combaciano con questo indirizzo; qualunque altra connessione verrà rifiutata. Questo viene fatto anche nel caso di un canale tramite ICMP. |

Tabella 4.2: Opzioni richieste nel comando del proxy

Per poter comunicare con l'attaccante, il proxy definisce un socket in cui rimane in ascolto. Nel caso si utilizzi il protocollo ICMP, monitorerà il flusso di rete per catturare i messaggi inviati dall'attaccante. Stabilita una comunicazione con esso, il proxy aspetterà un messaggio indicante l'indirizzo IP della vittima oltre alla tipologia di Covert Channel scelto.

I messaggi che un proxy può ricevere dall'attaccante sono:

1. Il messaggio indicante il comando (che il proxy dovrà inoltrare alla vittima) oppure se deve aspettare solo i dati (siccome non ha ricevuto il comando).
2. Il messaggio che indica la terminazione della comunicazione con l'attaccante. In questo caso il proxy aggiornerà la vittima della cosa.

Per la connessione con la vittima, il proxy comunicherà tramite pacchetti ICMP. Prima di inviare alcun dato, si imposta un thread con lo scopo di analizzare il traffico e catturare i dati che la vittima ritornerà. Se questo viene fatto dopo i pacchetti potrebbe andare persi. Una volta ricevuti tutti i dati dalla vittima, il proxy procederà ad inoltrarli all'attaccante.

### 4.1.3 Struttura Vittima

Per poterla inizializzare, la vittima richiede in input alcune opzioni (indicate nella Tabella 4.3).

| Opzione         | Utilizzo                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Numero di Proxy | Quando si eseguirà il programma, dovranno essere definiti il numero di proxy necessari. Ciò indicherà il numero minimo di proxy necessari che serviranno per l'esecuzione dell'attacco. Una volta raggiunto questo numero, la vittima procederà con l'esecuzione del comando. Altrimenti, se il numero non viene raggiunto, allo scadere del timer si chiederà se si vuole procedere comunque. |

Tabella 4.3: Opzioni richieste nel comando della vittima

La vittima rimane in attesa di connessioni da parte dei proxy. Ciò viene fatto monitorando il flusso di rete e filtrando i messaggi ICMP (destinati alla vittima) che rappresentano una richiesta di connessione. Se il messaggio ricevuto risulta valido, si invia al mittente un messaggio di conferma e il suo indirizzo IP viene inserito nella lista indicante i proxy connessi. Da questo messaggio la vittima ricaverà anche la metodologia di attacco scelto. E quindi ciascun proxy potrà usare (se voluto) una diversa tipologia di attacco.

Definiti i proxy con cui si comunicherà, si attenderà che inoltrino il comando dell'attaccante. Una volta ricevuto, verrà eseguito e i risultati ricavati (sia quelli legati all'output che quelli legati ad eventuali errori) vengono inviati all'attaccante tramite i proxy disponibili. Nel caso siano presenti molteplici proxy connessi, si definirà una lista indicante i dati che ciascuno di essi dovrà ricevere. L'ultimo messaggio che verrà inviato a ciascuno dei proxy sarà quello che indica che tutti i dati sono stati mandati.

## 4.2 Implementazione del Timing Covert Channel

Per poter temporizzare i dati da inviare, c'è bisogno sia di un metodo per la codifica e decodifica dei tempi. Dovrà quindi essere presente una funzione che è responsabile di mappare un dato binario a un intervallo di tempo.

Per la **codifica**, si calcolerà dal dato il delay associato; che verrà usato per indicare il



tempo da aspettare prima di inviare un nuovo pacchetto. Per esempio al dato binario 1001 potrebbe essere associato il tempo 3; quindi il programma, prima di mandare un nuovo pacchetto, aspetterà tre secondi.

Invece per la **decodifica**, il destinatario rimane in attesa dei pacchetti che gli vengono inviati. Ogni volta calcolerà l'intervallo di tempo fra un pacchetto e il successivo, e da quello ricaverà il dato che gli è associato. Per esempio se l'intervallo di tempo risultasse di 6 secondi, il destinatario controllerà a chi è associato tale tempo e ricaverà il dato binario 1101.

Per l'invio dei dati, si è preso spunto dall'approccio implementato in ICMP Exfill [17] nel quale il delay viene determinato dal valore del byte che si vuole esfiltrare. Ciò è possibile siccome un carattere di un testo viene memorizzato tramite una sequenza di bit la quale potrà essere interpretata anche come un numero intero.

### Funzione di codifica

In una **prima versione** (mostrata nella Figura 4.6), si associa ad ogni punto una distanza di sicurezza, coì da evitare che due codici combacino verso lo stesso tempo di delay. Di seguito la formula usata per calcolare il tempo di delay associato ad un byte e nella Tabella 4.4 la descrizione dei parametri usati:

$$\text{tempo\_base} + \text{indice} * \text{distanza\_tempi} \quad (4.1)$$

| Parametro            | Descrizione                                                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tempo di base</b> | Il minimo di secondi che si dovrà aspettare per ciascun possibile dato. È necessario per evitare di causare un overflow di pacchetti verso la macchina che riceve i dati. |
| <b>Indice</b>        | È l'indice associato alla codifica del dato. Il suo valore va da 0 sino a 255 siccome si prende un byte alla volta.                                                       |

|                             |                                                                               |
|-----------------------------|-------------------------------------------------------------------------------|
| <b>Distanza fra i tempi</b> | Distanza minima di secondi che tutte le codifiche dovranno avere fra di loro. |
|-----------------------------|-------------------------------------------------------------------------------|

Tabella 4.4: Parametri della prima funzione (Timing Channel)

Le distanze fra i vari intervalli di tempo sono necessarie per evitare errori nella decodifica. Infatti durante la trasmissione del pacchetto possono avvenire dei ritardi non costanti per tutti i datagram inviati [29][9]. La variazione del valore di questo parametro dipenderà dall'ubicazione geografica dell'entità mittente e di quella destinataria, oltre alla reattività delle due macchine.

La mediana di questo valore [19] è di solito 122 ms con un massimo di 266 ms<sup>4</sup>.

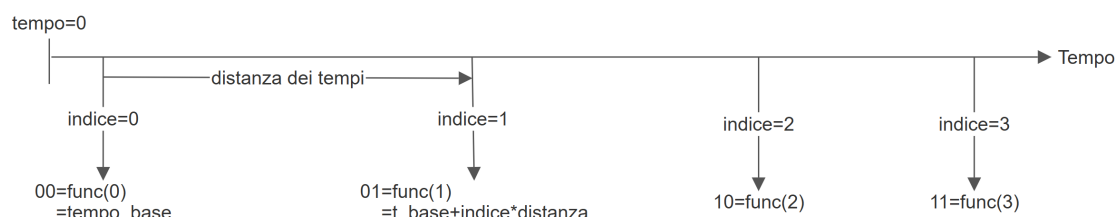


Figura 4.6: Assegnazione degli intervalli con la distanza

Lo svantaggio di questo approccio sono i tempi che si aspettano. Nel miglior caso si aspetterà solo il tempo di base nel caso peggiore invece 255 volte la distanza scelta.

Cercando di avere una stima più precisa, si sa che i caratteri stampabili vanno da 32 a 126[31]. Da ciò si ricava che in media si aspetteranno 79 secondi<sup>5</sup> per ciascun carattere. Supponendo che il tempo di base sia di 3 secondi e che la distanza fra i tempi sia di 0.6 secondi (600 ms); in media si aspetteranno circa 51 secondi<sup>6</sup>. È quindi **richiesta una seconda implementazione** per evitare tempi di attesa eccessivamente lunghi.

<sup>4</sup>I dati presi in analisi sono quelli relativi all'Italia

<sup>5</sup> $\frac{32+126}{2} = 79$

<sup>6</sup> $3 + \frac{32+126}{2} * 0.6 = 50.4$

Siccome il collo di bottiglia risultava il valore che lo stesso byte aveva, si è pensato di ridurre i dati trasmissibili riducendo i valori ASCII usati nella trasmissione. In particolare si saranno inviati solo i caratteri stampabili.

La scelta è stata ritenuta valida siccome nella codifica ASCII i principali caratteri stampabili vanno da 32 sino a 126 (come mostra la Tabella 4.5). Tuttavia ciò, oltre ad aggiungere overhead inutile, avrebbe tagliato fuori alcuni caratteri speciali (come il tab o il carattere di nuova linea). Inoltre il metodo sarebbe stato vincolato alla codifica ASCII. Quindi se i dati fossero stati codificati tramite un altro metodo, o si fosse mandato un file che non fosse di testo, sarebbero stati tagliati dati importanti.

| Range                                 | Descrizione                                                                                                                |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Control Character</b> [0-31]       | Codici di controllo non stampabili e che vengono utilizzati per controllare le periferiche.                                |
| <b>Printable character</b> [32-127]   | Rappresentano lettere, cifre, segni di punteggiatura e vari simboli.                                                       |
| <b>Extended ASCII Codes</b> [128-255] | Non fanno parte dell'ASCII standard ma alla sua versione estesa. I caratteri presenti dipendono dalla codifica utilizzata. |

Tabella 4.5: Struttura della tabella ASCII

Quindi si è definita una funzione che cercherà di normalizzare l'intervallo di tempo (associato al dato), fra un valore minimo e uno massimo[12][28][27]. Tramite questa funzione il range dei delay viene ristretto in un intervallo preciso e definito (come mostra la Figura 4.7).

$$\text{min\_delay} + (\text{byte}/255) * (\text{max\_delay} - \text{min\_delay}) \quad (4.2)$$

Tramite questo approccio; l'ampiezza dell'intervallo permetterà di regolare la velocità di trasmissione delle informazioni (che può essere maggiore o minore) con il

costo di avere una maggiore sensibilità ai delay fra l'invio e la ricezione dei pacchetti.

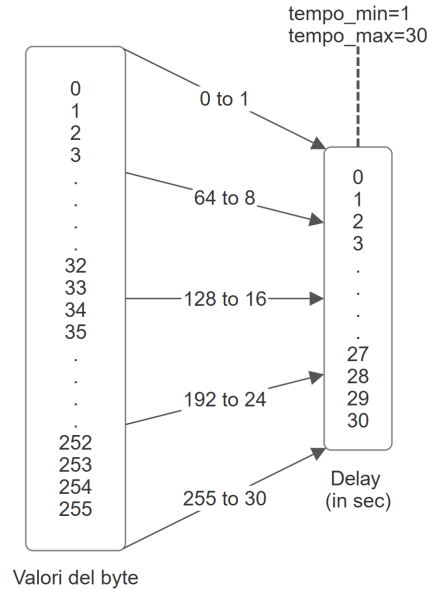


Figura 4.7: Normalizzazione degli intervalli (Timing Channel) [20]

#### 4.2.1 Randomizzazione dei tempi di invio

Un invio dei pacchetti con tempi costanti può essere facilmente individuato dagli IDS. Segue che uno schema che sembri randomico, permetterà di non essere rilevati. Si evolverà quindi il Covert Channel implementando del rumore. La Figura 4.8 mostra come il tempo di delay venga randomizzato aggiungendo del rumore.

Il tempo di delay associato a un byte non sarà più costante ma spazierà fra un range di valori, nel quale potrà variare. Questo porta che, per potersi scambiare i dati, il mittente e il destinatario devono potersi sincronizzare sulla quantità di rumore che si è aggiunto. Le strade possibili sono due:

1. L'entità mittente creerà un numero randomico e lo usa per calcolare il rumore. Le entità si dovranno quindi sincronizzare sul seed usato oltre al numero di estrazioni effettuate. Se i valori estratti non combaciassero; le entità non riusciranno a scambiarsi i dati correttamente.
2. Il mittente indica il rumore generato nel pacchetto stesso (per esempio in uno dei campi o il payload). Il ricevente otterrà il valore del rumore generato dalla

risorsa condivisa (il pacchetto). In questo caso non si avrà un Timing Channel puro, ma un Timing Channel ibrido.

3. Gli intervalli temporali non trasportano alcun dato. Ma in questo caso non si tratterà più di un Timing Covert Channel.

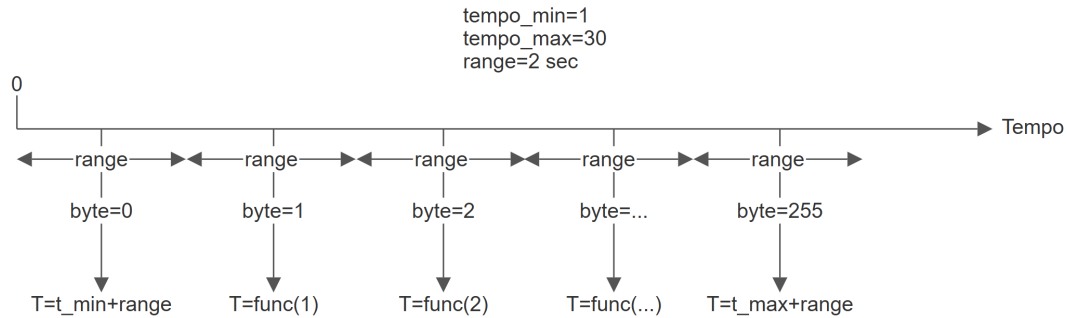


Figura 4.8: Randomizzazione dei tempi con il rumore (Timing Channel)

#### 4.2.2 Requisiti per la comunicazione

Siccome il destinatario rimane in ascolto e, all'arrivo di un pacchetto, ricava il dato associato all'intervallo di tempo; avrà bisogno di un messaggio che indichi quando la trasmissione inizia. Nel timing channel si dovrà quindi prevedere un modo per indicare l'inizio della comunicazione. Il metodo potrà essere una particolare sequenza inserita in un campo del pacchetto mentre oppure una particolare tipologia di messaggio ICMP (se non entrambi i metodi).

### 4.3 Implementazione del Storage Covert Channel

In uno Storage Covert Channel, la risorsa condivisa sarà il pacchetto ICMP inviato e i dati saranno scritti nei suoi campi. Il destinatario, una volta ricevuto, potrà poi leggere i valori codificati al loro interno. Nelle Tabelle 4.6 e 4.7 sono indicati le tipologie di messaggi sfruttate. Le tabelle indicano anche quali campi sono stati utilizzati e quanti byte pesa un singolo pacchetto.

Tuttavia, nel caso di alcune tipologie (come i messaggi Echo Request/Reply), si avranno delle varianti sia nella quantità di dati esfiltrabili sia sulla quantità di byte che

vengono trasmessi per singolo datagram. Ciò è dovuto alla presenza del campo *data* che permette di inserire una quantità maggiore di dati.

| Tipologia                 | Byte per pacchetto         | Campi Utilizzati                                             |
|---------------------------|----------------------------|--------------------------------------------------------------|
| Destination Unreachable   | 20+8+21<br>→min 49 byte    | unused(4 bytes)<br>header+64 bits (2bytes)                   |
| Time Exceeded             | 20+8+21<br>→min 49 byte    | unused(4 byte)<br>header+64 bits(2 byte)                     |
| Parameter Problem         | 20+8+21<br>→min 49 byte    | pointer(1byte)<br>unused(3byte)<br>header+64 bits(2byte)     |
| Source Quench             | 20+8+21<br>→min 49 byte    | unused(4byte)<br>header+64 bits(2byte)                       |
| Redirect Message          | 20+8+21=min 49 byte        | header+64 bits(2byte)                                        |
| Echo Request/Reply        | 20+8+32<br>→min 60 byte    | identifier(2byte)<br>data( $\geq 32$ )                       |
| Timestamp Request/Reply   | 20+8+12+32<br>→min 72 byte | identifier(2byte)<br>timestamp(4byte*3)<br>data( $\geq 32$ ) |
| Information Request/Reply | 20+8=28 byte               | identifier(2byte)                                            |

Tabella 4.6: Tipologie di messaggi ICMPv4 usati

| Tipologia               | Byte per pacchetto      | Campi Sfruttati                            |
|-------------------------|-------------------------|--------------------------------------------|
| Destination Unreachable | 40+8+41<br>→min 89 byte | unused(4 bytes)<br>invoking packet(2bytes) |
| Time Exceeded           | 40+8+41<br>→min 89 byte | unused(4 bytes)<br>invoking packet(2bytes) |
| Parameter Problem       | 40+8+41<br>→min 89 byte | pointer(4byte)<br>invoking packet(2byte)   |
| Echo Request/Reply      | 40+8+32<br>→min 80 byte | identifier(2byte)<br>data( $\geq 32$ )     |

|                |                         |                                      |
|----------------|-------------------------|--------------------------------------|
| Packet Too Big | 40+8+41<br>→min 89 byte | mtu(4byte)<br>invoking packet(2byte) |
|----------------|-------------------------|--------------------------------------|

Tabella 4.7: Tipologie di messaggi ICMPv6 usati

### 4.3.1 Inserimento dei dati nei campi usati

#### Campo unused

Dalle specifiche RFC (792 [6] e 4434 [5]) il campo deve essere 0. Quindi Scapy, quando manderà il pacchetto, lo invierà conforme allo standard e di conseguenza azzererà qualsiasi valore al suo interno (come mostra la Figura 4.9).

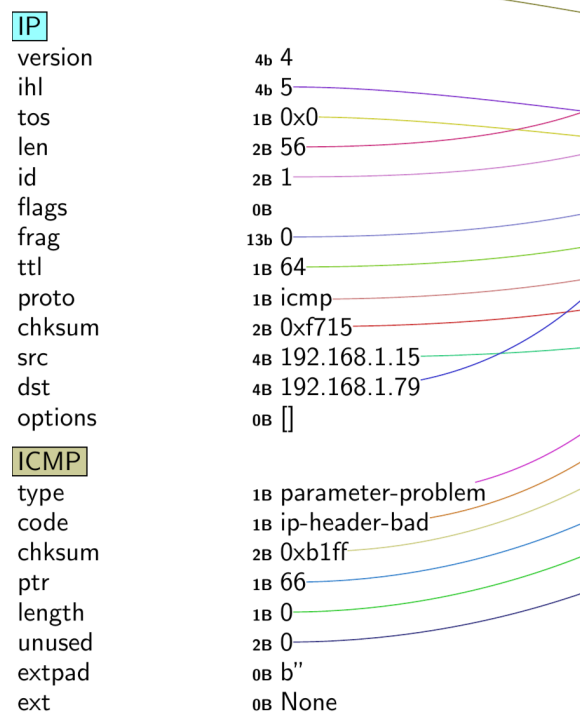


Figura 4.9: Messaggio IP/ICMP con campo *unused* azzerato

Il problema può essere risolto tramite degli accorgimenti. Tramite la libreria **struct** [22], È possibile inserire dei dati all'interno del campo. Infatti si può riscrivere, il pacchetto evitando di farlo creare Scapy. Così, quando lo si farà inviare da Scapy, al suo interno saranno presenti i dati. In Figura 4.10 si vede la loro presenza.

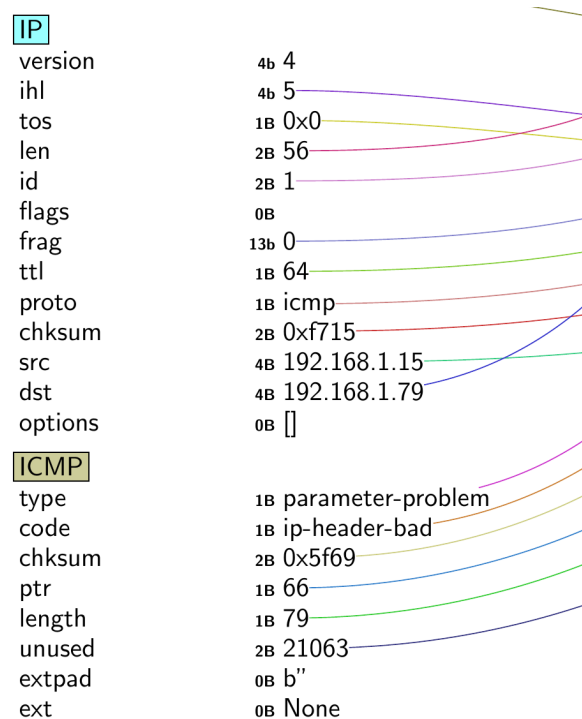


Figura 4.10: Messaggio IP/ICMP con campo *unused* impostato

Siccome il suo utilizzo rende il messaggio non conforme agli standard RFC [6][5]; un IDS che implementi la Deep Packet Inspection rileverà l'anomalia. Si potrà quindi scegliere se inserire dati nel campo o non.

### Campo Header+64 bits

Nel campo si dovranno inserire l'header IP più 8 byte. Si potranno usare quindi varie tipologie di pacchetti purchè la loro lunghezza non superi i 28 byte.

Inoltre siccome non rappresenta un datagram da inviare ma già spedito, si possono sfruttare un maggior numero di campi (indicati nella Tabella 4.8). La Figura 4.11 invece mostra un caso applicato ai protocolli IP e ICMP.

Header IPv4

| Campo        | Byte   | Utilizzo                    |
|--------------|--------|-----------------------------|
| Time to live | 1 byte | Tempo di vita del pacchetto |



|                |        |                                           |
|----------------|--------|-------------------------------------------|
| Total length   | 2 byte | Dimensione dell'intero pacchetto          |
| Identification | 2 byte | Identifica i frammenti di un pacchetto IP |

#### Header ICMPv4

| <b>Campo</b> | <b>Byte</b> | <b>Utilizzo</b>                                        |
|--------------|-------------|--------------------------------------------------------|
| Identifier   | 2 byte      | Identificativo del messaggio di richiesta invocato     |
| Sequence     | 2 byte      | Numero di sequenza del messaggio di richiesta invocato |

Tabella 4.8: Uso del Header+64 bits nel caso di IP/ICMP

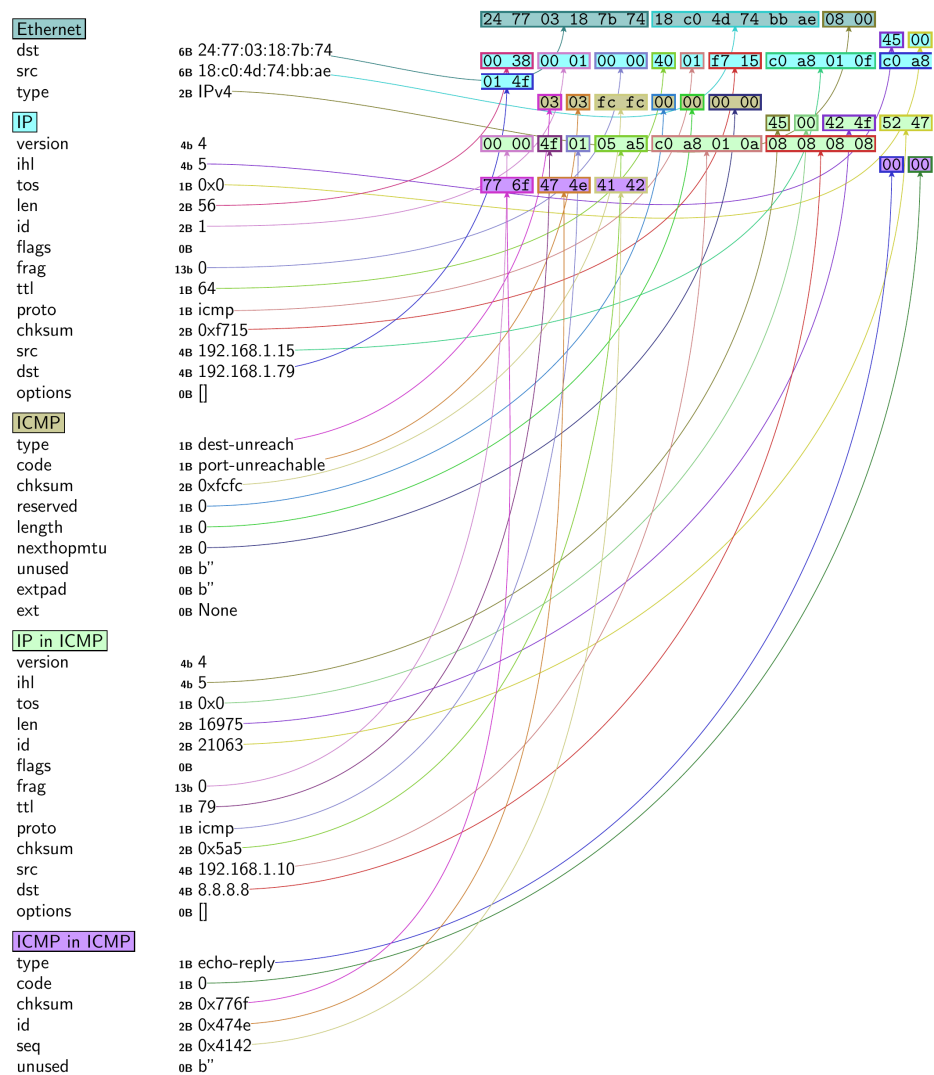


Figura 4.11: Messaggio IP/ICMP con campo *Header+64 bit* impostato

```

0000 24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00 $w...{t..Mt....E.
0010 00 38 00 01 00 00 40 01 F7 15 C0 A8 01 0F C0 A8 .8....@.....
0020 01 4F 03 03 68 66 42 4F 52 47 45 00 4F 47 4E 41 .0..hfBORGE.OGNA
0030 00 00 42 01 09 B3 C0 A8 01 0A 08 08 08 00 00 ..B.....
0040 69 5E 4F 52 47 4F i~ORGO
  
```

Codice 4.1: Dump esadecimale del messaggio in figura 4.11

### Campo Invoking Packet

Quando si indica il pacchetto che ha causato l'errore, si dovrà stare attenti a non superare la *IPv6 MTU*. Nel nostro caso, ciò non succederà siccome si useranno i protocolli IPv4 la cui intestazione è di 40 byte, ed il protocollo ICMPv4, la cui intestazione è di 8 byte. La Tabella 4.9 indica i campi utilizzati nel caso IPv6/ICMPv6.

| Header IPv6    |        |                                                          |
|----------------|--------|----------------------------------------------------------|
| Campo          | Byte   | Utilizzo                                                 |
| Payload Length | 2 byte | Tempo di vita del pacchetto                              |
| Hop Limit      | 1 byte | Decrementato di 1 per ogni nodo che inoltra il pacchetto |
| Header ICMPv6  |        |                                                          |
| Campo          | Byte   | Utilizzo                                                 |
| Identifier     | 2 byte | Identificativo del messaggio di richiesta invocato       |
| Sequence       | 2 byte | Numero di sequenza del messaggio di richiesta invocato   |

Tabella 4.9: Struttura del campo Invoking Packet

### Campo Pointer

Indica l'ottetto del messaggio in cui è presente l'errore; tuttavia può contenere dei dati non correlati ad esso. Nel nostro caso, verranno inseriti tanti dati quanto la sua capacità; come mostra la Figura 4.12.

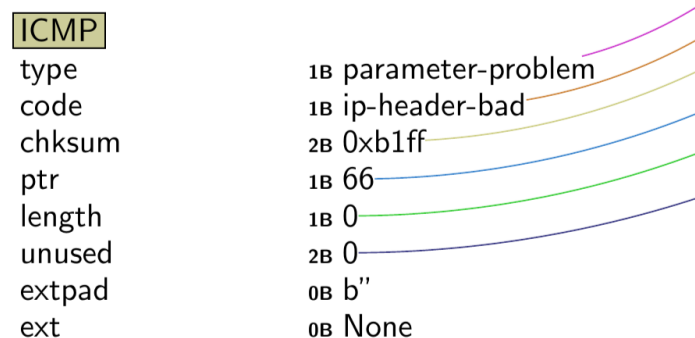


Figura 4.12: Messaggio IP/ICMP con campo *pointer* impostato

```

0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w...{t..Mt....E.
0010  00 38 00 01 00 00 40 01 F7 15 C0 A8 01 0F C0 A8  .8....@.....
0020  01 4F 0C 00 B1 FF 42 00 00 00 45 00 4F 52 47 4F  .0....B...E.ORG0
0030  00 00 47 01 0B 9A C0 A8 01 0A 08 08 08 08 00 00  ..G.....
0040  6F 6F 4E 41 42 4F                                ooNAB0

```

Codice 4.2: Dump esadecimale del messaggio in Figura 4.12

## Campo Identifier e Sequenza

Il campo *Identifier* definisce l'identificativo delle richieste e può assumere un qualsiasi valore. Il campo *Sequenza* (dalle specifiche RFC 792 [6]), viene incrementato ad ogni richiesta inviata con lo stesso Identifier. Se suo valore cambiasse troppo spesso la cosa potrebbe risultare sospetta (a differenza del campo *Identifier*). Quindi sebbene può essere utilizzato per inserire le informazioni, si userà quasi esclusivamente il campo Identifier (come mostra la Figura 4.13).

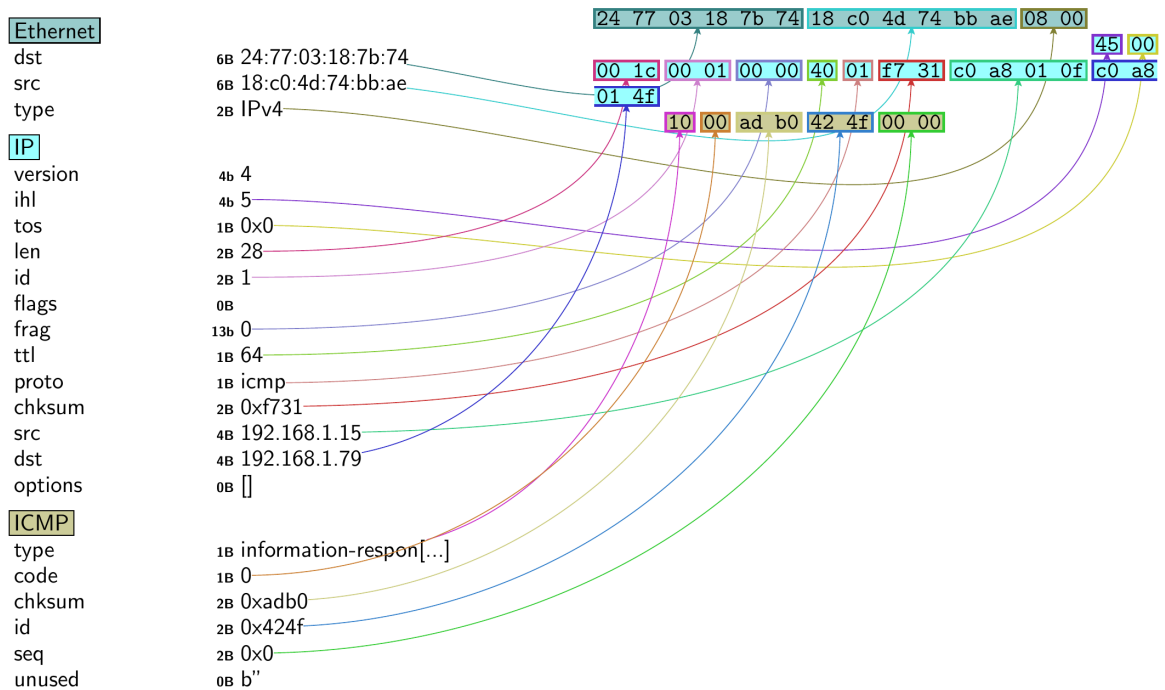


Figura 4.13: Messaggio IP/ICMP con campo *identifier* impostato

```
#PACCHETTO No1
0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w..{t..Mt....E.
0010  00 1C 00 01 00 00 40 01 F7 31 C0 A8 01 0F C0 A8  .....@..1.....
0020  01 4F 10 00 AD B0 42 4F 00 00                      .0....B0..

#PACCHETTO No2
0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w..{t..Mt....E.
0010  00 1C 00 01 00 00 40 01 F7 31 C0 A8 01 0F C0 A8  .....@..1.....
0020  01 4F 10 00 9D B8 52 47 00 00                      .0....RG..

#PACCHETTO No3
0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w..{t..Mt....E.
0010  00 1C 00 01 00 00 40 01 F7 31 C0 A8 01 0F C0 A8  .....@..1.....
0020  01 4F 10 00 A0 B8 4F 47 00 00                      .0....0G..

#PACCHETTO No4
0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w..{t..Mt....E.
0010  00 1C 00 01 00 00 40 01 F7 31 C0 A8 01 0F C0 A8  .....@..1.....
0020  01 4F 10 00 A1 BE 4E 41 00 00                      .0....NA..
```

Codice 4.3: Dump esadecimale del messaggio in Figura 4.13

Campo Data

In questo campo il mittente può inserire quanti dati preferisce. Ma per sicurezza la dimensione sarà o 32 o 64 byte. Inoltre una quantità di dati eccessiva può far mandare più pacchetti di risposta nel caso si fosse inviata una richiesta.

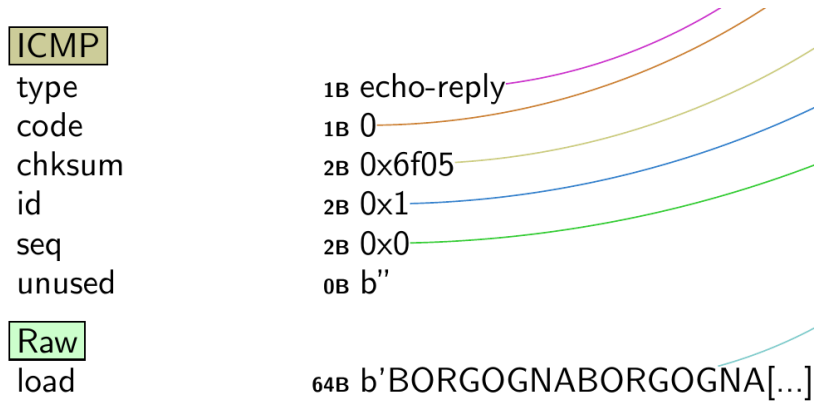


Figura 4.14: Messaggio IP/ICMP con campo *Data* impostato

```
0000 24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00 $w...{t..Mt....E.
0010 00 5C 00 01 00 00 40 01 F6 F1 C0 A8 01 0F C0 A8 .\....@.....
0020 01 4F 00 00 6F 05 00 01 00 00 42 4F 52 47 4F 47 .0...o.....BORGOG
0030 4E 41 42 4F 52 47 4F 47 4E 41 42 4F 52 47 4F 47 NABORGOGNABORGOG
0040 4E 41 42 4F 52 47 4F 47 4E 41 42 4F 52 47 4F 47 NABORGOGNABORGOG
0050 4E 41 42 4F 52 47 4F 47 4E 41 42 4F 52 47 4F 47 NABORGOGNABORGOG
0060 4E 41 42 4F 52 47 4F 47 4E 41 NABORGOGNA
```

Codice 4.4: Dump esadecimale del messaggio in Figura 4.14

Campo Timestamp

Contiene i millisecondi dalla mezzanotte UT e ciascun campo ha un ordine temporale specifico; che dovrà rimanere congruente quando si inseriscono i dati. Il Codice 4.5 mostra come in ciascun campo timestamp verrà inserito un byte nella parte dei millisecondi. [Figura 4.5].

```
Dato nascosto:  b'r '    114
Dato nascosto:  b'c '    99
Dato nascosto:  b'o '   111
```

```

Timestamp origin prima: 2025-11-07 23:13:36.146254+00:00
Timestamp origin dopo: 2025-11-07 23:13:36.114000+00:00
Tempo del campo: 83616114

```

Codice 4.5: Esempio di un dato inserito nel campo Timestamp

Non è possibile inserire ulteriori informazioni nelle altre parti del campo temporale: rappresentando le ore, i minuti ed i secondi. Infatti hanno dei valori definiti e limitati, come si vede dalla Figura 4.15 (che mostra il valore contenuto nei campi).

Si potrebbero inserire il dato in tutto il campo rappresentate il dato temporale; ma questo ci espone al rischio di avere tempi errati <sup>7</sup>.

Inoltre dalla Figura 4.15 si può constatare la presenza dell'informazione nella parte dei millisecondi. Dalle informazioni del pacchetto, i caratteri ricavati saranno le lettere R, G ed O; siccome i corrispettivi valori nella codifica ASCII [1] sono 82, 71 e 79.

| ICMP   |                    |
|--------|--------------------|
| type   | 1B timestamp-reply |
| code   | 1B 0               |
| chksum | 2B 0xb3            |
| id     | 2B 0x424f          |
| seq    | 2B 0x0             |
| ts_ori | 4B 22:42:16.82     |
| ts_rx  | 4B 22:42:17.71     |
| ts_tx  | 4B 22:42:18.79     |
| unused | 0B b''             |

Figura 4.15: Messaggio IP/ICMP con campi *Timestamp* impostati

---

<sup>7</sup>Per esempio l pacchetto viene risultare ricevuto prima che sia stato inviato.

```

#PACCHETTO No1
0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w...{t..Mt....E.
0010  00 28 00 01 00 00 40 01 F7 25 C0 A8 01 0F C0 A8  .(....@...%.....
0020  01 4F 0E 00 00 B3 42 4F 00 00 04 DF 31 92 04 DF  .0....B0....1...
0030  35 6F 04 DF 39 5F                                     5o...9_

#PACCHETTO No2
0000  24 77 03 18 7B 74 18 C0 4D 74 BB AE 08 00 45 00  $w...{t..Mt....E.
0010  00 28 00 01 00 00 40 01 F7 25 C0 A8 01 0F C0 A8  .(....@...%.....
0020  01 4F 0E 00 FB C9 47 4E 00 00 04 DF 31 81 04 DF  .0....GN....1...
0030  35 6A 04 DF 39 5F                                     5j...9_

```

Codice 4.6: Dump esadecimale del messaggio in Figura 4.15

## Campo MTU

Nel campo viene indicata la capacità massima del collegamento, Siccome il suo valore non è un valore prestabilito (ed è un intero variabile); l'intera dimensione del campo, potrà essere usato per inserire i dati.

## 4.4 Implementazione Behavioural Covert Channel

In questo caso ciò che indicherà l'informazione trasmessa, è l'arrivo di un determinato messaggio ICMP (la cui tipologia e codice indicano il dato). La Figura 4.16 mostra come gli eventi vengono mappati ai dati trasmessi.

Siccome la quantità di messaggi possibili risulta 17; ogni evento permetterà di codificare 4 bit alla volta<sup>8</sup>. L'evento che rimane inutilizzato, verrà usato per indicare l'inizio e la terminazione dell'invio dei dati.

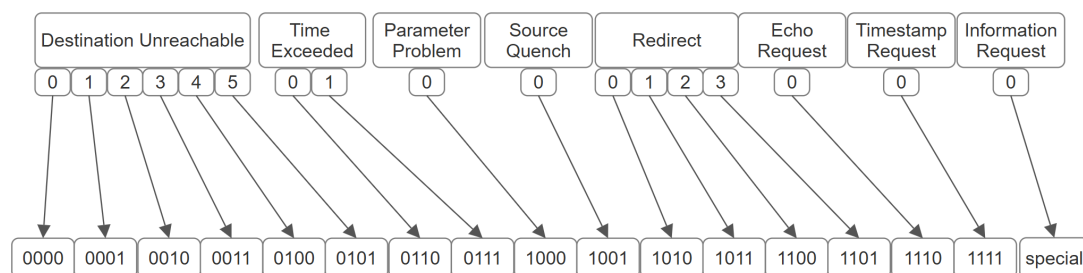


Figura 4.16: Schema Hybrid Channel [23]

<sup>8</sup> $\log_2 16 = 4$



Siccome un singolo evento può trasmettere solo 4 bit; l'invio di un byte, prevede la sua divisione in due parti. Quindi i primi quattro bit ricavati (così come gli ultimi quattro), determineranno la tipologia ed il codice del messaggio da inviare. Il Codice 4.7 mostra questo processo.

```
data='m'
data.hex=0x6D
data.bytes=b'0110 1101'
```

```
Maschera il byte con b'11110000' per ottenere: 0x60
Shift a destra di 4 bit per ottenere: 0x06
I primi 4 bit sono: 0110
```

```
Maschera il byte con b'00001111' per ottenere: 0x0D
Gli ultimi 4 bit sono: 1101
```

```
Mappa b'0110' alla tipologia ed al codice:
    Tipologia: Time Exceeded (tipo 11)
    Codice: 0 (Time to Live exceeded in Transit)
Mappa b'1101' alla tipologia ed al codice:
    Tipologia: Redirect (tipo 5)
    Codice: 3 (Redirect for the host)
```

```
Invia un messaggio ICMP Time Exceeded con codice 0
Invia un messaggio ICMP Redirect con codice 3
```

Codice 4.7: Esempio dell'invio di un byte tramite due eventi

Il destinatario, per ricavare i dati, monitora il traffico di rete e controllerà le tipologie di messaggi ICMP ricevuti; per ognuno di essi, prende la coppia (Tipologia, Codice) e da questa ricaverà i quattro bit associati.

## Tipologie di messaggi deprecati

Un problema di questa possibile implementazione, sono le tipologie deprecate dei messaggi. Questi messaggi potranno comunque essere inviati ma le linee guida [13][11][10] indicano ai router di ignorarli e all'host utente di non mandarli.

L'uso di tipologie deprecate renderà quindi il canale meno legittimo e più facile da individuare. Inoltre, siccome questo tipo di traffico sarà visto con sospetto, non è assicurata la ricezione dei messaggi.

## Source Quench

Il messaggio ICMP Source Quench (tipo 4, codice 0) era concepito come meccanismo per il controllo della congestione. Tuttavia, data la sua inefficacia per la congestione, la generazione di questi messaggi da parte dei router è stata formalmente deprecata dal 1995. Per questo la maggior parte delle implementazioni TCP ignorano silenziosamente i messaggi ICMP Source Quench. Infatti TCP implementa i propri meccanismi di controllo della congestione (che non dipendono dai messaggi ICMP Source Quench).

Quindi, siccome la reazione a questi messaggi nei protocolli di trasporto non è mai stata formalmente deprecata, la IETF (Internet Engineering Task Force) ha aggiornato le linee guida riguardo il messaggio Source Quench in questo modo:

- Un host **NON DEVE inviare** dei messaggi di questo tipo. Se viene ricevuto un messaggio di questo tipo, **PUÒ ignorarlo** (silenziosamente).
- Un router **DEVE ignorare** tutti i messaggi ICMP Source Quench ricevuti. Inoltre **NON DOVRÀ inviare** messaggi ICMP Source Quench in risposta a una congestione.
- Se un protocollo di trasporto riceve un messaggio Source Quench, **DEVE essere ignorato/scartato** silenziosamente.
- Host, gateway e firewall **DEVONO** scartare silenziosamente i pacchetti ICMP Source Quench ricevuti e **DOVREBBERO registrare** la cosa come un errore di sicurezza con almeno i seguenti dettagli (indirizzo IP sorgente, indirizzo IP

di destinazione, tipo di messaggio ICMP, data/ora in cui il pacchetto è stato visualizzato).

Nell'Internet attuale, non ci sono ragioni per cui un host debba generare o reagire a un messaggio ICMP Source Quench (o perchè un router debba reagire). L'unico motivo per cui un utente ignori questo requisito, è per poter consumare le risorse di rete. I messaggi ICMP Source Quench potrebbero essere infatti sfruttati per eseguire attacchi "blind throughput-reduction". Le linee guida indicano di ignorare silenziosamente questi messaggi; questo eliminerà il vettore di attacco.

Inoltre, siccome la generazione e la reazione ai messaggi ICMP Source Quench è stata deprecata, le applicazioni non dovranno aspettarsi di ricevere questi messaggi.

### **Information Request/Reply**

È stato deprecato siccome meccanismi come il DHCP [4], lo hanno sostituito per la configurazione dell'host

## **4.5 Implementazione di un Hybrid Covert Channel**

Un Covert Channel ibrido è stato implementato combinando le tre tipologie di Covert Channel usate precedentemente:

- *Timing Channel*: un byte indicherà il delay fra un pacchetto e l'altro.
- *Behavioural Channel*: da un byte verranno ricavate le due tipologie di messaggi da mandare.
- *Storage Channel*: in ciascun pacchetto verranno inseriti tanti dati quanto la sua capacità di trasmissione.

La comunicazione viene iniziata tramite un messaggio *Information Request* e chiusa sempre da un messaggio dello stesso tipo.

Il destinatario, per ricevere i dati, monitora il flusso dei pacchetti nella rete per rilevare l'intervallo di tempo con cui arrivano le coppie di messaggi. Da questo si ricaverà il byte relativo al tempo. Dalla coppia di messaggi ricava i quattro bit associati

a ciascun messaggio. Dall'unione dei bit ricaverà il byte relativo alla tipologia di messaggi inviati. Ora non resta che ricavare i dati che sono stati nascosti nei campi dei messaggi ICMP ricevuti.

```

Amministratore: Windows PowerShell
Data b'Dato mandato dal computer di Marco per testare un timing channel a 8 bit'
Ricavo MAC address: Get-NetNeighbor -IPAddress 192.168.1.17 | Where-Object {$_.State -eq 'Reachable' -or $_.State -eq 'S
tale'} | Select-Object -First 1 -ExpandProperty LinkLayerAddress
Tabella di routing non contiene MAC address per 192.168.1.17

Ricavo MAC address (Get-NetAdapter -InterfaceIndex (Get-NetIPAddress -IPAddress '192.168.1.17').InterfaceIndex).MacAddre
ss
MAC for 192.168.1.17:24-77-03-18-7B-74
In ascolto dei pacchetti ICMP...
Init      Type: 15 Start: None
ipv4_time_exceeded
Testo pacchetto: to man
ipv4_destination_unreachable
Testo pacchetto: dato dal
ipv4_time_exceeded
Testo pacchetto: ompute
ipv4_destination_unreachable
Testo pacchetto: r di Mar
ipv4_time_exceeded
Testo pacchetto: per t
ipv4_timestamp_reply
Testo pacchetto: estar
ipv4_destination_unreachable
Testo pacchetto: un timin
ipv4_destination_unreachable
Testo pacchetto: g channe
ipv4_destination_unreachable
Testo pacchetto: a 8 bit
End      Type: 15 Start: 1758665001.591177
Dati ricevuti: ['D', 'a', 'to man', 'dato dal', ' ', ' ', 'c', 'ompute', 'r di Mar', 'c', 'o', ' per t', 'estar', 'e', ' ', ' ',
un timin', 'g channe', 'l', 'a 8 bit']
Dati ricevuti: Dato mandato dal computer di Marco per testare un timing channel a 8 bit
Tempo init: 1758665001.591177 Tempo end: 1758665063.034469 Delta:
Tempo di esecuzione: 61.443291902542114
PS C:\Users\Marco\Desktop\Script tesi magistrale>

```

Figura 4.17: Ricezione dei dati tramite Hybrid Covert Channel

# Capitolo 5

## Test e Risultati

### 5.1 Test dei Covert Channel con RITA

In questi test a delle saranno presenti delle variazioni che sono:

- La quantità di dati inviati. In questo caso saranno 1KB, 10KB, 100KB, 1MB.
- I campi usati dal Covert Channel. In quest caso si usa il CC che sfrutta il protocollo ICMP Echo; le variazioni si baseranno sull'utilizzo o meno del campo Identifier o del payload. Nel caso si utilizzasse il payload, quest'ultimo potrà contenere una quantità di dati fissa o variabile per ogni pacchetto.
- Il delay fra i blocchi di dati. I dati verranno suddivisi in blocchi aventi una certa dimensione (e.g 100KB); si può decidere se avere un delay fra l'invio di un blocco e il successivo oppure un invio sequenziale.
- Tempo di attesa prima di un nuovo invio. Una volta che si sono inviati tutti i dati ci potrà essere un tempo di riposo prima di poter inviare nuovamente altri dati. Questo tempo sarà variabile (e.g 1 ora, 30 minuti) e dipenderà dalla quantità di dati esfiltrati (e.g 100KB, 1MB)
- Il mittente del messaggio. Può essere un singolo mittente che invierà tutti i pacchetti oppure si potrà falsificare il campo ed inserire valori diversi; così da far sembrare che il pacco sia stato spedito da un altro host.

L'obiettivo è determinare come RITA risponde alle comunicazioni con queste caratteristiche. Il Covert Channel utilizzato durante la fase di test è quello basato sul protocollo ICMP Echo.

### 5.1.1 Singolo invio di dati

In questa tipologia di test; i dati sono stati mandati una singola volta. I dati sono divisi in blocchi da 1024 bytes ( $\approx$  1KB). Invece il tempo di delay fra un blocco e il successivo, sarà uniforme fra 1 secondo e 15 secondi.

Nella Tabella 5.1 vengono riportate le etichette assegnate da RITA ad ogni comunicazione. In particolare si è utilizzato un Covert Channel che invia dati tramite ICMP Echo Request; in cui può utilizzare determinati campi del messaggio.

Un valore **None** equivale a dire che RITA ha inserito nel database la comunicazione ma che non gli ha assegnato alcun'etichetta.

Invece **Non presente** (o **NP**) vuol dire che la comunicazione non è presente nel database creato da RITA.

Infine **Low**, **Medium** e **High** sono le etichette assegnate da RITA sulla gravità della comunicazione

| Covert Channel                   | Quantità dei dati |              |              |              |
|----------------------------------|-------------------|--------------|--------------|--------------|
|                                  | 1 KB              | 10 KB        | 100 KB       | 1 MB         |
| Campo ID                         | Non presente      | Non presente | Non presente | Non presente |
| Campo ID + delay                 | Non presente      | Non presente | Non presente | Low          |
| Campo ID + payload fisso         | Non presente      | Non presente | Non presente | Non presente |
| Campo ID + payload fisso + delay | Non presente      | Non presente | Non presente | Low          |

|                           |              |              |              |              |
|---------------------------|--------------|--------------|--------------|--------------|
| Payload fisso             | Non presente | Non presente | Non presente | Non presente |
| Payload fisso + delay     | Non presente | Non presente | Non presente | Low          |
| Payload randomico         | Non presente | Non presente | Non presente | None         |
| Payload randomico + delay | Non presente | Non presente | Non presente | Low          |

Tabella 5.1: Test tramite un singolo invio dei dati

I risultati non mostrano una presenza di un Covert Channel siccome i dati sono stati mandati una singola volta nella giornata. Un successivo step è l'invio degli stessi dati molteplici volte.

### 5.1.2 Molteplici invii dei dati

In questi test si invierà per tre volte lo stesso dato variando alcune condizioni di invio. Le condizioni che potranno variare fra i vari test possono essere:

La **dimensione del dato inviato**. In questo caso i valori possibili sono 1MB e 100KB.

Il **tempo di attesa** dopo il quale si può di ritornare ad esfiltrare i dati. I tempi sono 1 ora, 30 minuti, 15 minuti.

Se usare un **delay fra un blocco di dati e il successivo**. Nel caso affermativo si invierà un blocco di una certa dimensione e poi si aspetta l'intervallo di tempo. Dopodichè si manda il blocco successivo. L'altro caso è l'invio dei dati sequenzialmente, senza pause.

Se **falsificare il mittente o no**. In questo caso i pacchetti non avranno l'IP della macchina vittima ma un indirizzo preso da un pool di indirizzi IP. Al suo

interno ci potranno essere o gli indirizzi IP degli host attualmente attivi sulla rete oppure gli indirizzi IP non utilizzati (e quindi non associati ad alcun host fisico).

Siccome nei test il Covert Channel esfiltrerà i dati tramite messaggi ICMP Echo; i dati verranno inseriti nel payload. Quest'ultimo, rispetto al campo Identifier, riesce a contenere una quantità maggiore di dati e questo porterebbe a meno pacchetti inviati. Infatti, se il canale venisse rilevato tramite l'utilizzo del payload, sicuramente verrà rilevato utilizzando solo il campo Identifier (siccome quest'ultimo ha una capacità ridotta rispetto al campo payload).

### **Test con payload fisso, invio di 1MB di dati e delay fra i blocchi di dati**

Analizziamo ora la Tabella 5.2: che indica un test in si sono mandati 1MB di dati, con un payload fisso, ed un delay fra un blocco di dati e il successivo.

Partiamo dall'etichetta che RITA ha assegnato a ciascuna comunicazione.

I test che hanno fatto uso del mittente reale hanno tutte etichette di livello *Medium*.

I test che hanno fatto uso di mittenti falsificati hanno principalmente etichette di livello *Low*; tranne un caso che risulta essere di livello *Medium*.

Si segue con l'analisi del Beacon Score.

Nel primo caso, quello con mittente reale, il Beacon Score più basso risulta essere quello in cui si aspetta 15 minuti. Mentre il massimo lo si ha nel caso in cui si aspetta 30 minuti. Nel caso 1 ora si ha un valore intermedio.

Nel secondo caso, quello con mittenti falsificati, il Beacon Score risulta decrescente all'aumentare del tempo di attesa.

Da ciò si deduce che tempi di attesa estremamente brevi e ed estremamente lunghi sono da preferire. Infatti nel caso con il mittente reale, un tempo di attesa di 15 minuti ha permesso di ottenere un Beacon Score di 0%; che vuol dire che RITA non



ha rilevato impulsi di omunicazione. Mentre nel caso in cui si è aspettato un ora, ciò è stato classificato come un impulso di comunicazione. Ciò spiegherebbe perchè il caso che aspetta 30 minuti ha lo score maggiore.

Nel caso con mittenti falsificati, il Beacon Score più basso si ha nel caso in cui si aspetta un ora. Mentre il valore maggiore si ha nel caso in cui si aspettano 15 minuti. Da cio si deduce che il cambio di mittenti senza un tempo di attesa adeguato, può essere visto da RITA come un impulso di comunicazione.

Ovviamente questo può anche essere influenzato dal fatto di ocme Zeek estrae i log che poi verranno fatti analizzare da RITA.

| Covert Channel                            | Connessione più lunga | Etichetta | Beacon Score |
|-------------------------------------------|-----------------------|-----------|--------------|
| Pausa di un ora e mittente reale          | 10h 3m 6s             | Medium    | 14.7%        |
| Pausa di 30 minuti e mittente reale       | 10h 4m 51s            | Medium    | 37.9%        |
| Pausa di 15 minuti e mittente reale       | 10h 9m 20s            | Medium    | 0%           |
| Pausa di un ora e mittenti falsificati    | 6h 52m 26s            | Low       | 0.0%         |
| Pausa di 30 minuti e mittenti falsificati | 6h 50m 55s            | Medium    | 13.5%        |
| Pausa di 15 minuti e mittenti falsificati | 4h 47m 22s            | Low       | 15.8%        |

Tabella 5.2: Test con payload fisso, 1MB di dati e un delay durante l'invio

### Test con payload fisso ed invio sequenziale di 1MB di dati

Nella Tabella 5.3 viene riportato il test in cui si sono mandati 1MB di dati, con un payload fisso, e senza un delay fra un blocco di dati e il successivo.

In questo caso sia che l'utente sia veritiero sia che i mittenti siano falsificati, RITA non riesce a rilevare la presenza di un Covert Channel. Ciò sembra più un problema relativo a come Zeek ha estratto i log piuttosto che a un mancato rilevamento da parte di RITA.

L'invio sequenziale dei dati sembra essere un metodo efficace per non essere rilevati; tuttavia ciò potrebbe degli squilibri sul flusso di rete. Finché la dimensione totale dei dati inviati risulta bassa non si sa se ciò non corrisponde ad un problema. Ma sicuramente un quantità elevata di dati inviati in questo modo risulta un chiaro indizio sulla presenza di un Covert Channel.

Siccome questo vale per tutti i test che non fanno uso del delay, i risultati che si sono ottenuti da essi, non verranno riportati. Infatti in tutti i test senza delay, RITA non ha rilevato la loro presenza.

| Covert Channel                      | Connessione più lunga | Etichetta | Beacon Score |
|-------------------------------------|-----------------------|-----------|--------------|
| Pausa di un ora e mittente reale    | NP                    | NP        | NP           |
| Pausa di 30 minuti e mittente reale | NP                    | NP        | NP           |
| Pausa di 15 minuti e mittente reale | NP                    | NP        | NP           |

|                                                     |          |      |     |
|-----------------------------------------------------|----------|------|-----|
| Pausa di un ora e mittenti falsificati <sup>1</sup> | 0h 1m 5s | None | 33% |
| Pausa di 30 minuti e mittenti falsificati           | NP       | NP   | NP  |

---

<sup>1</sup>il dato si riferisce ad una connessione TCP

|                                           |    |    |    |
|-------------------------------------------|----|----|----|
| Pausa di 15 minuti e mittenti falsificati | NP | NP | NP |
|-------------------------------------------|----|----|----|

Tabella 5.3: Test con payload fisso, 1MB di dati e nessun delay durante l'invio

### Test con payload fisso, invio 100 KB di dati e delay fra i blocchi di dati

Nella Tabella 5.4 viene riportato il test in cui si sono mandati 100KB di dati, con un payload fisso, e con un delay fra un blocco di dati e il successivo.

Come nel caso precedente, si nota come i test con mittente reale e mittenti falsificati non siano stati rilevati da RITA. Anche in questo si può pensare che il motivo sia da ricercare nella bassa quantità di dati inviati che potrebbero essere sufficienti per non essere rilevati.

| Covert Channel                                   | Connessione più lunga | Etichetta | Beacon Score |
|--------------------------------------------------|-----------------------|-----------|--------------|
| Pausa di un ora e mittente reale                 | NP                    | NP        | NP           |
| Pausa di 30 minuti e mittente reale              | NP                    | NP        | NP           |
| Pausa di 15 minuti e mittente reale <sup>1</sup> | 0h 0m 3s              | High      | 100%         |

|                                           |            |      |       |
|-------------------------------------------|------------|------|-------|
| Pausa di un ora e mittenti falsificati    | 0h 39m 42s | None | 30.4% |
| Pausa di 30 minuti e mittenti falsificati | NP         | NP   | NP    |

---

<sup>1</sup>il dato si riferisce ad una connessione TCP

|                                           |    |    |    |
|-------------------------------------------|----|----|----|
| Pausa di 15 minuti e mittenti falsificati | NP | NP | NP |
|-------------------------------------------|----|----|----|

Tabella 5.4: Test con payload fisso, 100KB di dati e un delay durante l'invio

Siccome ciò vale anche per un altro test, i cui risultati sono presenti nella Tabella 5.5, si ritiene l'ipotesi valida. In particolare il test invia 100KB di dati con un payload variabile e con un delay fra un blocco di dati e il successivo.

| <b>Covert Channel</b>               | <b>Connessione più lunga</b> | <b>Etichetta</b> | <b>Beacon Score</b> |
|-------------------------------------|------------------------------|------------------|---------------------|
| Pausa di un ora e mittente reale    | NP                           | NP               | NP                  |
| Pausa di 30 minuti e mittente reale | NP                           | NP               | NP                  |
| Pausa di 15 minuti e mittente reale | NP                           | NP               | NP                  |

|                                           |    |    |    |
|-------------------------------------------|----|----|----|
| Pausa di un ora e mittenti falsificati    | NP | NP | NP |
| Pausa di 30 minuti e mittenti falsificati | NP | NP | NP |
| Pausa di 15 minuti e mittenti falsificati | NP | NP | NP |

Tabella 5.5: Test con payload randomico, 100KB di dati e un delay durante l'invio

## Test con payload variabile, invio di 1MB di dati e delay fra i blocchi di dati

Nella Tabella 5.4 viene riportato il test in cui si è inviato 1MB di dati, con un payload variabile e con un delay fra un blocco di dati e il successivo.

Partiamo dal test con il mittente reale:

In questo caso RITA riesce a rilevare la presenza del Covert Channel ed assegna un'etichetta di livello *Medium* al test che aspetta un ora mentre l'etichetta *Low* a quello che aspetta 30 minuti.

Per quanto riguarda il Beacon Score, assegna un valore pari a 0% al test che aspetta un ora; mentre un valore di 25% a quello che aspetta 30 minuti. Vedendo i risultati anche degli altri test, si ritiene che il valore più basso sia un errore di calcolo da parte di RITA. Siccome gli ha assegnato anche una gravità maggiore

Passiamo ora ai test che falsificano il mittente. In questo caso si sono fatti due varianti del test: una usa gli indirizzi IP assegnati ad host attivi nella rete mentre l'altra usa indirizzi IP non assegnati ad alcun host. Siccome il test è stato fatto in una rete domestica; sono pochi gli indirizzi IP assegnati (in una rete con una subnet /24). Ciò porterà ad avere un numero maggiore di indirizzi IP non assegnati rispetto a quelli assegnati.

I risultati sono i seguenti:

Nel caso si usi gli host attivi, RITA assegna a quasi tutti i test un'etichetta di livello *Low*; con un Beacon Score del 32.8%

Nel caso in cui si utilizzino indirizzi IP non assegnati, RITA assegna un livello di gravità pari a *Low* ed un Beacon Score pari al 79%.

Una differenza che si può notare è la durata massima della connessione. nel primo caso risulta di circa 7 ore (in linea con il caso del mittente reale) mentre nel caso degli IP non assegnati, risulta di circa 1 ora. Ciò è dovuto alla quantità di indirizzi IP usati nel secondo caso.

Le conclusioni sono che un payload variabile e randomico no migliora la discrezione del Covert Channel. Infatti RITA assegna a tutti i test i un Beacon Score che va dal 25% al 79%. Inoltre la quantità di indirizzi IP usabili può portare ad un peggioramento del canale. Questo se non vengono divisi in batch ed usati ciclicamente; in quest'ultimo caso potrebbero portare ad un miglioramento .

| Covert Channel                      | Connessione più lunga | Etichetta | Beacon Score |
|-------------------------------------|-----------------------|-----------|--------------|
| Pausa di un ora e mittente reale    | 10h 4m 6s             | Medium    | 0%           |
| Pausa di 30 minuti e mittente reale | 6h 49m 12s            | Low       | 25%          |

I mittenti falsificati provengono da un pool di indirizzi IP attivi nella rete locale

|                                        |            |     |       |
|----------------------------------------|------------|-----|-------|
| Pausa di un ora e mittenti falsificati | 6h 52m 13s | Low | 32.8% |
|----------------------------------------|------------|-----|-------|

I mittenti falsificati provengono da un pool di indirizzi IP non assegnati nella rete locale

|                                        |           |     |     |
|----------------------------------------|-----------|-----|-----|
| Pausa di un ora e mittenti falsificati | 1h 3m 15s | Low | 79% |
|----------------------------------------|-----------|-----|-----|

Tabella 5.6: Test con payload randomico, 1MB di dati e un delay durante l'invio

### 5.1.3 Considerazioni sui risultati ottenuti

Dai risultati ottenuti si nota che l'utilizzo di un singolo mittente per l'esfiltrazione dei dati aumenta la probabilità di essere rilevati da parte di strumenti come RITA. Come si può vedere dalla Tabella 5.2 e dalla Tabella 5.6, con qualunque periodo di pausa la gravità assegnata risulta genralmente media.

Invece tramite l'utilizzo di mittenti falsificati il livello di gravità si abbassa. Come la

Tabella 5.2 mostra.

Tuttavia ciò ha dei difetti se non implementati appropriamente. La Tabella 5.6 mostra che il tasso di beaconing peggiora gravemente procedendo con l'utilizzo dei mittenti falsificati. Ciò è dovuto ad un'implementazione poco strutturata ed accorta della cosa. Siccome per ogni pacchetto si sceglie sempre un mittente nuovo (che può combaciare o no con quello prima). Un implementazione in cui per ogni blocco si utilizza un singolo mittente scelto dal pool, risulterebbe migliore.

Per quanto riguarda l'uso o meno del delay fra l'invio di un blocco di dati ed il successivo; è risultato che il suo non utilizzo permetta di non essere rilevati da RITA. Infatti per ogni test senza delay nel database di RITA non risultava niente.

Ciò può essere dovuto al fatto che il traffico generato risulti così veloce da non superare la soglia di rilevamento. Tuttavia questo comporta un maggior utilizzo della banda di rete. Ciò crea rumore da parte del Covert Channel siccome il traffico potrebbe modificare il normale flusso di rete.

# Capitolo 6

## Conclusioni

### Conclusioni sull'implementazione dei Covert Channel

Questo lavoro di tesi ha affrontato lo sviluppo e l'implementazione di un covert channel basato sul protocollo ICMP per l'estrazione di dati. L'obiettivo principale era dimostrare come il protocollo ICMP, fondamentale per il funzionamento delle reti ma spesso sottovalutato dal punto di vista della sicurezza, possa essere sfruttato per creare canali di comunicazione nascosti efficaci.

Durante lo sviluppo sono state implementate diverse tipologie di covert channel:

- Storage Covert Channel implementato sfruttando i vari campi dei messaggi ICMP.
- Timing Covert Channel codificando le informazioni negli intervalli temporali tra i pacchetti. Per poter inviare un maggior numero di informazioni (possibilmente senza renderlo maggiormente sensibile ai errori), si sono sviluppate varie varianti per il calcolo degli intervalli di tempo.
- Behavioural Covert Channel che codifica i dati attraverso l'invio di una specifica tipologia di messaggio e codice ICMP.
- Un implementazione ibrida dei Covert Channel precedentemente fatti.



Alcune difficoltà sono state riscontrate durante l'implementazione della comunicazione fra le varie entità. Non potendo affidarsi a TCP, ma solo a messaggi ICMP, si sono dovuti gestire metodi per stabilire e rilasciare la connessione stabilita. Oltre all'accettarsi che la comunicazione sia effettivamente stabilita.

Ulteriori difficoltà tecniche sono stati l'invio di messaggi non conformi agli standard RFC. Si sono dovute trovare alternative per poter inserire dati dove da linee guida non è possibile e gestire la presenza di tipologie di messaggi ICMP deprecate. Inoltre siccome l'attacco deve poter essere eseguibile anche tramite IPv6; si sono dovuti implementare metodi per la ricerca dell'interfaccia da utilizzare per l'invio dei dati.

### **Conclusioni sui test effettuati**

Nella trasmissione che invia una singola volta i dati, RITA non ha rilevato la presenza di un covert Channel. Ma questo caso non è applicabile alla realtà; in cui l'attaccante e la vittima si possono scambiare più volte i dati. Si è quindi proceduto a testare molteplici invii dei dati.

Le conclusioni sono che l'utilizzo di un delay fra l'invio di un blocco di dati ed il successivo permette di abbassare la visibilità del Covert Channel. Anche la falsificazione dei mittenti permette questa cosa, a patto che gli indirizzi IP utilizzabili non vengano utilizzati in sequenza per ogni pacchetto inviato ma a blocchi. Così da poter permettere agli IP precedentemente utilizzati di abbassare la loro visibilità.

In sintesi la rilevabilità di un Covert channel dipende da:

- Frequenza di trasmissione: Trasmissioni sporadiche sono più difficili da rilevare rispetto a comunicazioni con pattern regolari.
- Distribuzione delle sorgenti: L'utilizzo di mittenti multipli (reali o falsificati intelligentemente), permette di ridurre significativamente la rilevabilità.
- Intervalli irregolari e sufficientemente lunghi tra le trasmissioni riducono la probabilità di essere identificati come beaconing

## **Conclusioni sull'utilizzo di ICMP**

Sebbene sia un protocollo funzionale al monitoraggio di rete, i suoi messaggi devono essere limitati a quelli essenziali. In particolare le principali tipologie di messaggi ICMP che servono sono la tipologia Echo Request/Reply (usata in strumenti come ping), la tipologia Destination Unreachable e la tipologia Time Exceeded. Il resto delle tipologie sono state deprecate siccome sostituite da strumenti diversi.

Inoltre fra quelli autorizzati bisogna monitorare il loro utilizzo ed il loro contenuto. Un messaggio Echo Request può contenere dati nel campo payload e messaggi come Destination Unreachable o Time Exceeded possono generare dei pattern temporali.

Ciò richiede un'ispezione dei pacchetti presenti nel flusso, un'analisi statistica per la ricerca di pattern e la limitazione (nella rete) di messaggi ICMP.

# Bibliografia

- [1] ascii code.com. Ascii table reference to ascii table of windows-1252. URL <https://www.ascii-code.com/>.
- [2] Active Countermeasures. Rita, 2025. URL <https://github.com/activecm/rita>.
- [3] DhavalKapil. icmptunnel, 2025. URL <https://github.com/DhavalKapil/icmptunnel>.
- [4] Ralph Droms. Dynamic Host Configuration Protocol. RFC 2131, March 1997. URL <https://www.rfc-editor.org/info/rfc2131>.
- [5] RFC Editor. Rfc 4443, March 2006. URL <https://www.rfc-editor.org/rfc/rfc4443>.
- [6] RFC Editor. Rfc 792, September 1981. URL <https://www.rfc-editor.org/rfc/rfc792>.
- [7] Muawia Elsadig. Network covert channels. In *from the Edited Volume: Steganography - The Art of Hiding Information*, Joceli Mayer, Submitted: 09 March 2024 Reviewed: 11 March 2024 Published: 03 April 2024.
- [8] Gowthamraj F. Understanding the icmp protocol with wireshark in real time, Jan 21, 2022. URL <https://learningnetwork.cisco.com/s/article/Understanding-the-ICMP-Protocol-with-Wireshark-in-Real-Time>.
- [9] GeeksforGeeks. Delays in computer network, 28 Dec, 2024. URL <https://www.geeksforgeeks.org/computer-networks/delays-in-computer-network/>.

- [10] Fernando Gont. Deprecation of ICMP Source Quench Messages. RFC 6633, May 2012. URL <https://www.rfc-editor.org/info/rfc6633>.
- [11] Fernando Gont and Carlos Pignataro. Formally Deprecating Some ICMPv4 Message Types. RFC 6918, April 2013. URL <https://www.rfc-editor.org/info/rfc6918>.
- [12] Google. Dati numerici: normalizzazione, 2025-07-10 UTC. URL [https://developers.google.com/machine-learning/crash-course/numerical-data/normalization?hl=it#summary\\_of\\_normalization\\_techniques](https://developers.google.com/machine-learning/crash-course/numerical-data/normalization?hl=it#summary_of_normalization_techniques).
- [13] IANA. Internet control message protocol (icmp) parameters, 2025-04-29. URL <https://www.iana.org/assignments/icmp-parameters/icmp-parameters.xhtml>.
- [14] Md Nazmul Islam, Vinay Patil, and Sandip Kundu. Determining proximal geo-location of iot edge devices via covert channel. pages 196–202, 03 2017. doi: 10.1109/ISQED.2017.7918316.
- [15] Science Direct; Mazurczyk Wojciech; Tan Yu'an; Guri Mordechai; J.M. Keller. Covert channel. URL <https://www.sciencedirect.com/topics/computer-science/covert-channel>.
- [16] krabelize. icmpdoor, 2025. URL <https://github.com/krabelize/icmpdoor>.
- [17] martinoj2009. Icmpexfill, 2025. URL <https://github.com/krabelize/icmpdoor>.
- [18] M. Mecarelli. Sviluppo di un covert channel tramite il protocollo icmp, 2025. URL <https://github.com/mcrmr/Tesi-Magistrale---M.-Mecarelli-F.-Santini.git>.
- [19] Microsoft. Azure network round-trip latency statistics, 08/18/2025. URL <https://learn.microsoft.com/en-us/azure/networking/azure-network-latency?tabs=Americas%2CWestUS>.

- [20] WallStreet Mojo. Normalization formula, Unknown. URL <https://www.wallstreetmojo.com/normalization-formula/>.
- [21] Oracle Corporation. Virtualbox official website, 2025. URL <https://www.oracle.com/virtualization/virtualbox/>.
- [22] Python. struct — interpret bytes as packed binary data, Nov 07, 2025. URL <https://docs.python.org/3/library/struct.html>.
- [23] F. Santini.
- [24] SecDev. Scapy, 2025. URL <https://github.com/secdev/scapy>.
- [25] SecDev. Scapy, 2025. URL <https://scapy.readthedocs.io/en/latest/api/scapy.layers.inet.html>.
- [26] SecDev. Scapy, 2025. URL <https://scapy.readthedocs.io/en/latest/usage.html#sniffing>.
- [27] StackExchange. Map real numbers into [0:255] using fixed limits interval, Aug 2, 2012. URL <https://gamedev.stackexchange.com/questions/33441/how-to-convert-a-number-from-one-min-max-set-to-another-min-max-set>.
- [28] StackExchange. Map real numbers into [0:255] using fixed limits interval, Aug 28, 2015. URL <https://math.stackexchange.com/questions/1412371/map-real-numbers-into-0255-using-fixed-limits-interval>.
- [29] Wikipedia. Network delay, 18 October 2025. URL [https://en.wikipedia.org/wiki/Network\\_delay](https://en.wikipedia.org/wiki/Network_delay).
- [30] Wikipedia. Internet control message protocol, 2025. URL [https://en.wikipedia.org/wiki/Internet\\_Control\\_Message\\_Protocol](https://en.wikipedia.org/wiki/Internet_Control_Message_Protocol).
- [31] Wikipedia. List of unicode characters, 27 September 2025. URL [https://en.wikipedia.org/wiki/List\\_of\\_Unicode\\_characters](https://en.wikipedia.org/wiki/List_of_Unicode_characters).